

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»
Кафедра интеллектуальных информационных технологий

ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ В
ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ

Методические указания
по выполнению расчётной работы
2 курс

Минск 2025

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ.....	2
ОБЩИЕ СВЕДЕНИЯ.....	3
ВАРИАНТЫ ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ.....	59

ОБЩИЕ СВЕДЕНИЯ

Цель

Изучить принципы разработки ostis-систем, включая принципы разработки баз знаний и решателей задач ostis-систем.

Задачи

1. Изучить основные принципы и способы применения Технологии OSTIS.
2. Изучить основные принципы разработки ostis-систем и их компонентов.
3. Приобрести навыки разработки компонентов базы знаний и решателя задач ostis-систем.
4. Приобрести навыки решения комплексных задач при помощи Технологии OSTIS.

Теоретические сведения

1. Технология OSTIS

Технология OSTIS (Open Semantic Technology for Intelligent Systems) — это открытая семантическая технология проектирования интеллектуальных систем.

Целью *Технологии OSTIS* является создание систем, способных легко обучаться решению новых задач за счет интеграции различных типов знаний и моделей решения задач. *Технология OSTIS* направлена на решение проблем повторного использования компонентов интеллектуальных систем, обеспечивая их семантическую совместимость.

Технология OSTIS — это не конкретная интеллектуальная система или метод решения задач какого-либо класса, а технология разработки интеллектуальных систем, каждая из которых, в свою очередь, будет решать задачи определенного класса.

Ключевые преимущества OSTIS заключаются не в принципиально новых функциональных возможностях разрабатываемых систем (большинство функций ostis-систем можно реализовать с помощью традиционных средств), а в том, насколько легко модифицировать и развивать разрабатываемые системы, адаптировать их к новым задачам, а также насколько эффективно можно накапливать и использовать полученные компоненты при разработке новых систем, сокращая при этом время и трудоемкость их разработки.

Технология OSTIS — это способ решения проблемы совместимости, одной из важнейших проблем современных технологий.

1.1. Основные принципы Технологии OSTIS

Основными принципами Технологии OSTIS являются:

- ориентация на смысловое однозначное представление знаний в виде семантических сетей, имеющих базовую теоретико-множественную интерпретацию, что обеспечивает решение проблемы многообразия форм представления одного и того же смысла, и проблемы неоднозначности семантической интерпретации информационных конструкций;
- использование ассоциативной графодинамической памяти;
- применение агентно-ориентированной модели обработки знаний;
- обеспечение в проектируемых системах высокого уровня гибкости, стратифицированности, рефлексивности, гибридности и, как следствие, обучаемости.

Дополнительную информацию о Технологии OSTIS можно получить [здесь](#).

1.2. Понятие ostis-системы

ostis-система — это интеллектуальная система, разработанная по принципам Технологии OSTIS.

Каждая ostis-система состоит из следующих компонентов:

- *базы знаний,*
- *решателя задач*
- *и пользовательского интерфейса.*

База знаний ostis-системы может описывать любой вид знаний, при этом ее легко дополнять новыми видами знаний.

Решатель задач ostis-системы основан на многоагентном подходе и позволяет легко интегрировать и комбинировать любые модели решения задач.

Пользовательский интерфейс ostis-системы представляет собой подсистему со своей базой знаний и решателем задач.

Каждый из компонентов может быть полностью представлен на *SC-коде* — универсальном формальном языке унифицированного смыслового

представления знаний. В таком случае, её sc-модель (модель ostis-системы, представленная на SC-коде) может быть интерпретирована Платформой интерпретации ostis-систем — *Программной платформой ostis-систем*.

1.3. Понятие Программной платформы ostis-систем

Программная платформа ostis-систем — это программная платформа, реализующая интерпретацию информации в ostis-системах, обеспечивающая их компонентное проектирование, платформенную независимость, а также семантическую совместимость в рамках Экосистемы OSTIS.

Программная платформа ostis-систем состоит из:

- *программной реализации sc-памяти* — общей семантической памяти для хранения и обработки текстов на SC-коде;
- *программного интерфейса, обеспечивающего доступ к sc-памяти* — набора методов для создания, изменения, удаления и поиска sc-конструкций в sc-памяти;
- интерпретаторов различных подязыков SC-кода (*Языка SCP, Языка SCL* и других), обеспечивающих универсальность и платформенную независимость ostis-систем;
- а также средств разработки ostis-систем (*sc-сборщика базы знаний ostis-систем* и других).

2. Проект OSTIS

Проект OSTIS — это проект с открытым исходным кодом (открытый проект), ориентированный на разработку и продвижение *Технологии OSTIS*.

Проект OSTIS-AI был создан в марте 2022 года и является открытым проектом, функционирующим на платформе GitHub, целью которого является разработка инструментов для создания интеллектуальных систем по принципам *Технологии OSTIS*. Проект доступен по ссылке <https://github.com/ostis-ai/>.

Основной задачей данного проекта является непрерывное обеспечение эффективной среды для поддержки жизненного цикла:

- *Программной платформы ostis-систем*;
- *Стандарта OSTIS* и *Метасистемы OSTIS*;
- а также средств поддержки их разработки.

Проект OSTIS включает множество репозиторий, основными из которых являются:

1. Репозиторий *ostis-web-platform* — проект Платформы OSTIS, который предоставляет инструменты для установки и разработки *Платформы OSTIS*.
2. Репозиторий *ostis-metasytem* — проект *Метасистемы OSTIS*, представляющей собой комплексную интеллектуальную систему, предназначенную для автоматизации и управления всеми этапами жизненного цикла ostis-систем.
3. Репозиторий *ostis-example-app* — проект примера ostis-системы, который включает базовые возможности ostis-систем.

3. Принципы разработки ostis-системы путём модификации и расширения примера ostis-системы

В качестве базового проекта разработки рекомендуется использовать проект примера ostis-системы, который [доступен в открытом репозитории на GitHub](#). Данный проект представляет собой исходный код примера ostis-системы, на основе которого можно получить новую ostis-систему посредством изменения, замещения существующих компонентов и добавления новых компонентов.

Проект примера ostis-системы служит практической отправной точкой для понимания и использования *Технологии OSTIS*.

Для установки, конфигурации и запуска проекта примера ostis-системы необходимо следовать инструкции, указанным в [README-файле проекта на GitHub](#).

Ключевыми компонентами данного репозитория являются:

- директория *knowledge-base/* — содержит исходники базы знаний ostis-системы;
- директория *problem-solver/* — содержит исходники решателя задач ostis-системы;
- директория *interface/* — содержит исходники пользовательского интерфейса ostis-системы.

3.1. База знаний примера ostis-системы

База знаний примера ostis-системы, как и любой другой ostis-системы, представляется в виде *исходных файлов (исходников)*. Для представления информации в *исходных файлах базы знаний* используются *SCg-код* и *SCs-код*.

3.1.1. Понятие исходного файла базы знаний

исходный файл базы знаний — это файл с расширением *.gwf* или *.scs*, содержащий некоторый фрагмент базы знаний на *SCg-коде* или *SCs-коде*.

исходный файл базы знаний с расширением .gwf — это исходный файл, содержащий некоторый фрагмент базы знаний на *SCg-коде*.

исходный файл базы знаний с расширением .scs — это исходный файл, содержащий некоторый фрагмент базы знаний на *SCs-коде*.

3.1.2. Понятие фрагмента базы знаний

фрагмент базы знаний — это логически выделяемое *sc-знание* от других *sc-знаний* в базе знаний, которое может быть независимо формализовано, протестировано и интегрировано в общую структуру базы знаний.

3.1.3. Принципы тестирования фрагментов базы знаний

Процесс тестирования формализованных фрагментов предметной области направлен на проверку соответствия их синтаксиса и семантики правилам формализации фрагментов базы знаний. При этом тестирование следует рассматривать как обязательный этап цикла разработки базы знаний, обеспечивающий согласованность новых компонентов с уже существующими элементами.

3.1.3.1. Сборка и пересборка базы знаний *ostis-системы*

Для проверки синтаксической корректности формализованного фрагмента базы знаний требуется выполнить процесс *сборки всей базы знаний* с этим новым фрагментом.

сборка (пересборка) базы знаний ostis-системы — это трансляция множества заданных исходных файлов базы знаний во множество бинарных файлов базы знаний, которые представляют собой форму кодирования внутреннего представления информации в базе знаний *ostis-системы* и служат основой для ее эффективного хранения, доступа и последующей обработки в этой *ostis-системе*.

3.1.3.1.1. Понятие бинарного файла базы знаний

бинарный файл базы знаний — это бинарный файл, кодирующий информацию на *SC-коде*.

3.1.3.1.2. Понятие *sc*-сборщика базы знаний *ostis*-системы

Для сборки (пересборки) базы знаний *ostis*-системы следует использовать *sc*-сборщик базы знаний *ostis*-системы.

sc-сборщик базы знаний *ostis*-системы — это программное средство для сборки базы знаний *ostis*-системы, являющееся компонентом Программной платформы *ostis*-систем.

3.3.1.1.3. Принципы использования *sc*-сборщика базы знаний *ostis*-системы

sc-сборщик базы знаний *ostis*-системы компилируется отдельным независимым исполняемым бинарным файлом. В проекте примера *ostis*-системы его можно использовать следующим образом:

```
cd ostis-example-app/  
./install/sc-machine/bin/sc-builder -i repo.path -o kb.bin --clear
```

Опция `-i` (`--input`) позволяет указать либо путь к папке с исходными файлами базы знаний, либо путь к файлу конфигурации базы знаний, задающему пути тех исходных файлов базы знаний, которые следует собирать, и пути тех исходных файлов базы знаний, которые не следует собирать.

Файл конфигурации базы знаний в проекте примера *ostis*-системы — `repo.path` — содержит следующую информацию:

```
# components  
  
knowledge-base/  
  
!knowledge-base/ims.ostis.kb/ui/menu/ui_main_menu.scs  
  
# specifications  
  
problem-solver/cxx/example-module/specifications/agent_of_isomorphic_search  
  
problem-solver/cxx/example-module/specifications/agent_of_subdividing_search
```

Данную информацию можно проинтерпретировать как “собрать все исходные файлы из директории `knowledge-base/`, за исключением исходного файла `knowledge-base/ims.ostis.kb/ui/menu/ui_main_menu.scs` и собрать исходные файлы `problem-solver/cxx/example-module/specifications/agent_of_isomorphic_search` и `problem-solver/cxx/example-module/specifications/agent_of_subdividing_search`”.

В файле конфигурации базы знаний можно указывать как относительные пути, так и абсолютные пути к файлам.

Опция `-o (--output)` позволяет указать путь к папке, в которую будут компилироваться бинарные файлы базы знаний, то есть в которой в результате сборки исходных файлов базы знаний появятся бинарные файлы базы знаний.

Флаг `--clear` позволяет запустить `sc`-сборщик базы знаний `ostis`-системы в режиме полной переинициализации, при котором все ранее существующие бинарные файлы в директории `kb.bin` удаляются и заново формируются на основе заданных исходных файлов базы знаний.

Если выполнить сборку базы знаний ostis-систем без указания флага `--clear`, тогда существующие бинарные файлы в директории `kb.bin` будут дополнены указанными исходными файлами базы знаний.

После запуска sc-сборщика следует дождаться его завершения. Если в процессе сборки исходных файлов базы знаний появится ошибка с текстом красного цвета, то это значит, что какой-то исходный файл базы знаний составлен некорректно и имеют грамматические и синтаксические ошибки. Пример ошибки показан ниже:

[illegible]

Сборка базы знаний прервалась на 1380м исходном файле из 1397ми исходных файлов.

Если исходные файлы базы знаний составлены корректно, тогда sc-сборщик базы знаний ostis-системы завершит трансляцию исходных файлов успешно (без ошибок), а это значит, что полученные бинарные файлы базы знаний можно использовать для запуска ostis-системы.

Пример успешно собранной базы знаний демонстрируется на картинке ниже:

```
concept_of_reusable_component_interfaces.scs - ok
[1387/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
kb_editing/ui_menu_file_for_changing_main_identifier/lib_component_ui_menu_file_for_changing_main_identi
fier.scs - ok
[1388/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
na_activity_automatisation/ontology_forming/ui_menu_na_ontology_forming_content.scs - ok
[1389/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_fio.scs - ok
[1390/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_student.scs
- ok
[1391/1397]: knowledge-base/ims.ostis.kb/ui/model/ui_descr_oper_semantic_control.scs - ok
[1392/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
search/authorized_users/without_context/find_key_concepts/ui_menu_file_for_finding_key_concepts_content.
scs - ok
[1393/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/activity_automatisation/expert/assign_form_verif_prod/agent_assign_form_verif_pr
od_content.scs - ok
[1394/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
search/unauthorized_users/with_context/find_all_output_const_pos_arc_with_rel_in_the_agreed_part_of_kb/l
ib_component_ui_menu_view_all_output_const_pos_arc_with_rel_in_the_agreed_part_of_kb.scs - ok
[1395/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/Methods_evaluation/Kb_volume_metrics/agent_of_calculation_number_of_concepts_and
_relations/agent_of_calculation_number_of_concepts_and_relations_content.scs - ok
[1396/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_group.scs -
ok
[1397/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/verification/scp_program_component/section_library_of_reusable_scp_programs_for_
verification_agents.scs - ok
Clean state...
Statistics
Nodes: 25781(6.621%)
Connectors: 351351(90.233%)
Links: 12250(3.14601%)
Total: 389382
[17:24:56][Info]: Shutdown ScKeynodes
[17:24:56][Info]: [sc-memory] Shutdown
[17:24:56][Info]: [sc-memory] Shutdown extensions
[17:24:56][Info]: [sc-memory] Extensions shutdown
[17:24:56][Info]: [sc-memory] Shutdown sc-helper
[17:24:56][Info]: [sc-fs-memory] Save sc-memory segments
[17:24:56][Info]: Loaded segments count: 6
[17:24:56][Info]: Sc-segments size: 25165872
[17:24:56][Info]: Last not engaged segment num: 0
[17:24:56][Info]: Last released segment num: 6
[17:24:56][Info]: [sc-fs-memory] Sc-memory segments saved
[17:24:56][Info]: [sc-fs-memory] Save sc-fs-memory dictionaries
[17:24:56][Info]: [sc-fs-memory] Dictionary `term - offsets` written
[17:24:56][Info]: [sc-fs-memory] Dictionary `string offsets - link hashes` written
[17:24:56][Info]: Last string offset: 1136938
[17:24:56][Info]: [sc-fs-memory] All sc-fs-memory dictionaries saved
[17:24:56][Info]: [sc-fs-memory] Shutdown
[17:24:56][Info]: [sc-fs-memory] Successfully shutdown
[17:24:56][Info]: [sc-memory] Shutdown context manager
[17:24:56][Info]: [sc-memory] Shutdown
```

исходный файл решателя задач — это файл с расширением .hprp или .hpr, содержащий часть кода агента этого решателя задач на языке программирования C++.

3.1.3.2. Запуск ostis-системы

После успешной сборки базы знаний *ostis-системы* можно запустить *ostis-систему*.

запуск ostis-системы — это запуск Программной платформы *ostis-систем* вместе с компонентами заданной *ostis-системы*: базой знаний, решателем задач и пользовательским интерфейсом этой *ostis-системы*.

Для запуска примера *ostis-системы* необходимо запустить в разных терминалах:

- *sc-машину*:

```
cd ostis-example-app  
./scripts/start.sh machine
```

- *sc-веб*:

```
cd ostis-example-app  
./scripts/start.sh web
```

3.1.3.2.1. Понятие *sc-машины*

sc-машина — это программное средство, представляющее собой программную реализацию *sc-памяти* и программных интерфейсов для работы с ней.

При запуске *sc-машины* могут появиться ошибки с текстом в жёлтом или красном цветах. Это значит, что скорее всего после такого запуска *sc-машины* *ostis-система* будет работать неправильно. Следовательно, перед началом работы с *ostis-системой* необходимо устранить ошибки, возникающие при запуске *sc-машины*.

3.1.3.2.2. Понятие *sc-веба*

sc-веб — это реализация веб-ориентированного интерпретатора пользовательского интерфейса *ostis-системы*, взаимодействующего с *sc-машиной*.

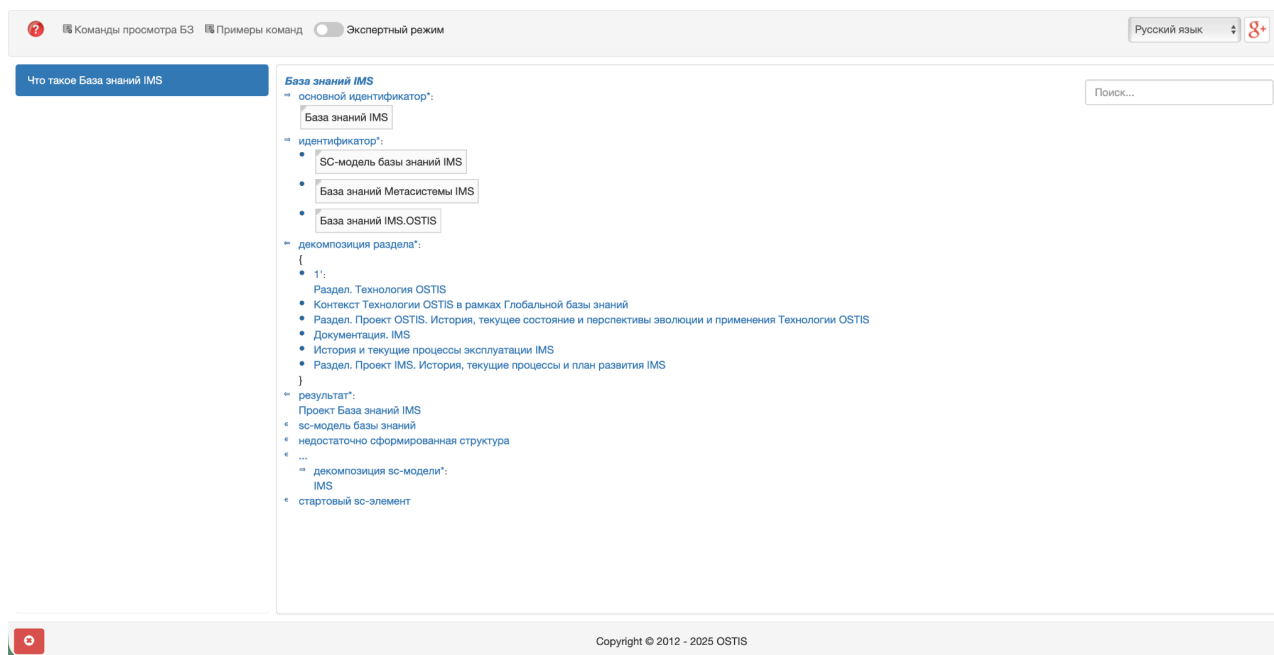
3.1.3.2.3. localhost:8000

sc-машина и *sc-веб* являются серверными приложениями. Это значит, что после запуска они должны “висеть” в терминалах, то есть эти терминалы нельзя закрывать в процессе работы с *ostis-системой*.

Чтобы остановить работу приложений в терминалах, необходимо в каждом терминале применить комбинацию клавиш `Ctrl+C` (`Control+C`).

После успешного запуска обоих приложений следует перейти в браузер и указать в адресной строке `localhost:8000`.

После загрузки появится следующая страница:



На главной части данной страницы показана семантическая окрестность сущности *База знаний IMS*, представленная на SCn-коде. С помощью данного интерфейса можно осуществлять *навигацию по базе знаний ostis-системы*.

4. Принципы разработки ostis-системы с нуля

При необходимости можно создать свой проект ostis-системы с нуля. Для этого необходимо выполнить инициализацию проекта, которая описывается далее.

4.1. Инициализация проекта ostis-системы

инициализация проекта ostis-системы — это процесс, включающий создание базовой структуры директорий компонентов этой ostis-системы: её базы знаний, решателя задач и пользовательского интерфейса, а также создание файлов конфигурации сборки, запуска и развёртывания этой ostis-системы.

На этом этапе создаются все необходимые директории и конфигурационные файлы, которые определяют, как будут собираться, запускаться и развёртываться компоненты ostis-системы.

4.1.1. Определение структуры проекта ostis-системы

Для начала необходимо создать базовую структуру проекта ostis-системы. Структура проекта новой ostis-системы может выглядеть следующий образом:

```
my-ostis-system/      # Проект ostis-системы
|-- knowledge-base    # Исходные файлы базы знаний (.scs, .gwf)
|-- problem-solver    # Исходные файлы решателя задач (агентов на
|                      C++)
|-- CMakeLists.txt    # Конфигурация сборки C++ агентов и их
|                      зависимостей
|-- conanfile.txt     # Спецификация менеджера пакетов Conan для
|                      установки зависимостей C++ агентов
```

4.1.2. Конфигурация зависимостей для C++ агентов с помощью менеджера пакетов Conan

В файле *conanfile.txt* указываются библиотеки, которые будут использоваться C++ агентами разрабатываемой ostis-системы (то есть библиотеки, необходимые для сборки и запуска исходных файлов C++ агентов), а также настройки сборки C++ агентов при помощи системы сборки CMake.

Другими словами, файл *conanfile.txt* задаёт внешние зависимости для C++ агентов.

conanfile.txt используется менеджером пакетов *Conan*.

Conan — это менеджер пакетов для C/C++, который упрощает управление зависимостями проекта, позволяя избежать ручной загрузки и настройки библиотек.

Файл *conanfile.txt* может содержать следующую информацию:

```
[requires]
sc-machine/0.10.5    # Указывает, что проект требует библиотеку
                    # sc-machine версии 0.10.5
gtest/1.14.0        # Указывает, что проект требует библиотеку
                    # gtest версии 1.14.0

[generators]
CMakeDeps            # Генератор для создания файлов информации
                    # о зависимостях для CMake
CMakeToolchain       # Генератор для создания файла цепочки
```

```

# инструментов для CMake

[layout]
cmake_layout      # Задаёт стандартную схему размещения
                  # дерева сборки проекта

```

В процессе разработки решателя задач *ostis*-систем данный файл может быть дополнен информацией о новых библиотеках, необходимых для его сборки и запуска. По мере добавления новых агентов или расширения функциональности системы могут потребоваться дополнительные библиотеки, которые легко добавляются в секцию `[requires]`.

4.1.3. Конфигурация CMake для *ostis*-системы

В файле *CMakeLists.txt* указываются настройки сборки для C++ агентов. Этот файл используется системой сборки *CMake*.

CMake — это кроссплатформенная система автоматизации сборки, которая генерирует файлы проектов для различных инструментов компиляции (*Make*, *Ninja*, *VSCode* и др.). Использование *CMake* обеспечивает переносимость проекта между различными операционными системами и средами разработки.

```

# Минимально требуемая версия CMake
cmake_minimum_required(VERSION 3.24)

# Установите стандарт C++17
set(CMAKE_CXX_STANDARD 17)

# Определение проекта
project(my-ostis-system VERSION 0.10.0 LANGUAGES C CXX)

# Конфигурация политики версий
cmake_policy(SET CMP0048 NEW)

# Конфигурация выходных каталогов
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
set(SC_EXTENSIONS_DIRECTORY
    ${CMAKE_LIBRARY_OUTPUT_DIRECTORY}/extensions)

# Поиск пакета sc-machine для реализации агентов
find_package(sc-machine REQUIRED)

```

```
# Поиск пакета GTest для тестирования агентов
include(GoogleTest)
find_package(GTest REQUIRED)

# Включение директории решателя задач
add_subdirectory(${CMAKE_CURRENT_SOURCE_DIR}/problem-solver)
```

Директива *cmake_minimum_required* указывает минимальную версию *CMake*, необходимую для корректной сборки проекта. Это обеспечивает доступность всех требуемых возможностей *CMake*.

Переменная *CMAKE_CXX_STANDARD* устанавливает стандарт языка C++ (в данном случае C++17). Это гарантирует, что компилятор будет использовать современные возможности языка, необходимые для разработки агентов.

Команда *project()* определяет имя проекта, его версию и используемые языки программирования. Это базовая информация о проекте, которая может использоваться другими частями конфигурации.

Переменные *CMAKE_RUNTIME_OUTPUT_DIRECTORY* и *CMAKE_LIBRARY_OUTPUT_DIRECTORY* определяют, куда будут помещаться скомпилированные исполняемые файлы и библиотеки. Переменная *SC_EXTENSIONS_DIRECTORY* указывает специальную директорию для расширений для *sc-машины* — скомпилированных модулей агентов.

Команда *find_package()* ищет установленные библиотеки и делает их доступными для использования в проекте. Опция *REQUIRED* означает, что сборка завершится с ошибкой, если библиотека не найдена. Это современная практика *CMake*, которая предпочтительнее ручного указания путей к библиотекам.

Команда *add_subdirectory()* подключает к сборке содержимое указанной директории. В данном случае это каталог *problem-solver*, который должен содержать собственный файл *CMakeLists.txt* с правилами сборки агентов (см. далее).

4.1.4. Установка основных инструментов разработки C++ агентов

Далее перед началом работы с *ostis*-системами необходимо установить базовый набор инструментов разработки. Эти инструменты обеспечивают возможность компиляции программ, управления зависимостями и

автоматизации сборки. Установка инструментов различается на различных операционных системах.

Чтобы установить эти инструменты на операционные системы *Ubuntu* или *Debian*, необходимо выполнить в терминале следующие команды:

```
sudo apt update
sudo apt install --yes --no-install-recommends \
    curl \
    ccache \
    python3 \
    python3-pip \
    build-essential \
    ninja-build
```

Чтобы установить эти инструменты на *macOS*, необходимо выполнить в терминале следующие команды:

```
brew update && brew upgrade
brew install \
    curl \
    ccache \
    cmake \
    ninja
```

Для других дистрибутивов *Linux*, которые не поддерживают *apt*, необходимо с помощью стандартного менеджера пакетов установить следующие пакеты:

- *curl* — утилита командной строки для передачи данных по различным сетевым протоколам (*HTTP*, *HTTPS*, *FTP* и др.). Используется для загрузки файлов и пакетов из интернета.
- *ccache* — кэш компилятора, который существенно ускоряет повторную компиляцию исходного кода. Он сохраняет результаты предыдущих компиляций и повторно использует их, когда исходный код не изменился.
- *python3* и *python3-pip* — интерпретатор языка *Python* версии 3 и менеджер пакетов *pip*. *Python* необходим для работы различных инструментов сборки и автоматизации, включая *Conan*.
- *build-essential* — мета-пакет, который содержит основные инструменты для компиляции программ на C и C++. Он включает компилятор *GCC* (для C), компилятор *G++* (для C++), утилиту *Make* для управления процессом сборки, библиотеки разработки *GNU C* (*libc6-dev*) и инструменты для работы с пакетами *Debian* (*dpkg-dev*).

- *ninja-build* — альтернативная система сборки, которая работает быстрее традиционной утилиты *Make*. *Ninja* специально разработана для максимальной скорости сборки больших проектов.

4.1.5. Установка менеджера пакетов Conan

Для установки менеджера пакетов *Conan* рекомендуется использовать *pipx* — инструмент для установки *Python*-приложений в изолированные окружения.

Чтобы установить *pipx*, рекомендуется выполнить команды, следуя официальной инструкции: <https://pipx.pypa.io/stable/installation/>.

pipx позволяет устанавливать *Python*-приложения глобально, но в изолированных виртуальных окружениях, что предотвращает конфликты между зависимостями различных приложений. Каждое приложение, установленное через *pipx*, получает собственное изолированное окружение с необходимыми ему библиотеками.

Также перед установкой *Conan* необходимо проверить версию установленного *CMake* и в случаях, если он не установлен, или если его версия < 3.24, выполнить в терминале следующие команды:

```
pipx install cmake  
pipx ensurepath
```

Команда *pipx ensurepath* добавляет директорию *~/.local/bin* (где *pipx* устанавливает приложения) в переменную окружения *PATH*, что позволяет запускать установленные приложения из любого места в операционной системе.

После можно установить менеджер пакетов *Conan*. Для этого необходимо выполнить в терминале следующие команды:

```
pipx install conan  
pipx ensurepath  
exec $SHELL
```

Команда *exec \$SHELL* перезапускает текущую оболочку командной строки, применяя изменения в *PATH* без необходимости закрывать и открывать терминал заново.

4.1.6. Установка sc-машины

После установки Conan необходимо настроить доступ к репозиторию пакетов *OSTIS-AI* и установить пакет *sc-машины*.

sc-машина — это программное средство, представляющее собой программную реализацию *sc*-памяти и программных интерфейсов для работы с ней (в том числе интерфейсов для разработки агентов на C++). Она является ядром любой *ostis*-системы, обеспечивая хранение и обработку знаний в форме семантических сетей.

Для настройки доступа к репозиторию пакетов *OSTIS-AI* необходимо в терминале в корне проекта выполнить следующие команды:

```
conan remote add ostis-ai \  
https://conan.ostis.net/artifactory/api/conan/ostis-ai-library  
conan profile detect
```

Команда *conan remote add* регистрирует новый удалённый репозиторий пакетов. Репозиторий *OSTIS-AI* содержит специализированные библиотеки для разработки *ostis*-систем, которые недоступны в общедоступных репозиториях.

Команда *conan profile detect* автоматически определяет конфигурацию вашей системы (операционную систему, компилятор, архитектуру) и создаёт профиль по умолчанию. Профиль используется *Conan* для выбора правильных бинарных версий библиотек или для их сборки из исходников.

Для установки пакеты *sc-машины* из репозитория *OSTIS-AI* с помощью *Conan* необходимо в терминале в корне проекта выполнить следующую команду:

```
conan install . --build=missing
```

Эта команда читает файл *conanfile.txt* в текущей директории, загружает указанные в нём зависимости и генерирует файлы для интеграции с *CMake*. Опция *--build=missing* указывает *Conan* собрать библиотеки из исходников, если готовые бинарные версии для вашей конфигурации системы недоступны.

4.1.7. Установка инструментов и расширений *sc-машины*

Скомпилированная *sc-машина* представляет собой пакет библиотек, которые можно установить при помощи *Conan* (см. выше), а также набор инструментов (исполняемых файлов) и расширений.

Инструментами *sc-машины* являются:

- бинарный файл, с помощью которого происходит сборка базы знаний *ostis-системы* — *sc-builder* (*sc-сборщик базы знаний ostis-системы*);
- и бинарный файл, с помощью которого происходит запуск *ostis-системы* — *sc-machine*.

Расширениями *sc-машины* считаются библиотеки базовых C++ агентов для *ostis-систем*.

Процесс установки инструментов и расширений *sc-машины* отличается между Linux и macOS. Для их установки на Linux, необходимо:

1. Загрузить готовый архив с инструментами и расширениями *sc-машины*, выполнив в терминале:

```
curl \
-LO
https://github.com/ostis-ai/sc-machine/releases/download/0.10.5/sc-
machine-0.10.5-Linux.tar.gz
```

Команда `curl` с опцией `-L` следует по перенаправлениям *HTTP*, а опция `-O` сохраняет файл с его оригинальным именем.

2. Распаковать загруженный архив в папку в *sc-machine*, выполнив в терминале:

```
mkdir sc-machine && tar -xvzf sc-machine-0.10.5-Linux.tar.gz -C
sc-machine --strip-components 1
```

Команда `tar` с опциями `-x` (`extract` — извлечь), `-v` (`verbose` — показывать процесс), `-z` (`gzip` — распаковать сжатие) и `-f` (`file` — имя файла) извлекает содержимое архива. Опция `-C` указывает целевую директорию для извлечения, а `--strip-components 1` удаляет один верхний уровень директорий из архива, помещая содержимое непосредственно в *sc-machine*.

3. Удалить ненужные исходники, выполнив в терминале:

```
rm -rf sc-machine-0.10.5-Linux.tar.gz && rm -rf sc-machine/include
```

После распаковки удаляются загруженный архив и заголовочные файлы (include), которые были нужны только при компиляции библиотек из исходников, но не требуются для работы с готовыми бинарными файлами.

Для установки инструментов и расширений *sc-машины* на macOS, необходимо:

1. Загрузить готовый архив с инструментами и расширениями *sc-машины*, выполнив в терминале:

```
curl \
-LO
https://github.com/ostis-ai/sc-machine/releases/download/0.10.5/sc-machine-0.10.5-Darwin.tar.gz
```

Darwin — это кодовое имя операционной системы, на которой основана macOS.

2. Распаковать загруженный архив в папку в *sc-machine*, выполнив в терминале:

```
mkdir sc-machine && tar -xvzf sc-machine-0.10.5-Darwin.tar.gz -C
sc-machine --strip-components 1
```

3. Удалить ненужные исходники, выполнив в терминале:

```
rm -rf sc-machine-0.10.5-Darwin.tar.gz && rm -rf
sc-machine/include
```

После выполнения всех этих шагов в проекте разрабатываемой *ostis-системы* будет полностью настроено окружение для разработки *ostis-системы* с нуля, включающее все необходимые инструменты, библиотеки и платформы.

4.2. Разработка базы знаний *ostis-системы*

база знаний — это систематизированная совокупность знаний, хранящаяся в памяти *ostis-системы* и достаточная для обеспечения целенаправленного (целесообразного, адекватного) функционирования (поведения) этой *ostis-системы* как в своей внешней среде, так и в своей внутренней среде (в собственной базе знаний).

База знаний любой ostis-системы содержит знания, используемые в процессе решения задач этой ostis-системой, при этом она может быть дополнена новыми знаниями в процессе решения этих задач.

4.2.1. Создание структуры базы знаний ostis-системы

Под разработкой базы знаний ostis-системы прежде всего понимается разработка исходных файлов фрагментов этой базы знаний. Все исходные файлы базы знаний ostis-системы обычно разрабатываются в директории *knowledge-base/*.

В качестве примера формализуем фрагмент *Предметной области множеств*.

Структура директорий и исходных файлов с формализацией заданной предметной области может выглядеть следующим образом:

```
knowledge-base/
|-- repo.path                # Файл конфигурации исходных
|                             файлов базы знаний
|-- subject-domain-of-sets/  # Директория предметной области
|   |-- subject_domain_of_sets.scs # Исходный файл предметной
|   |                             области
|   |-- concepts/            # Понятия предметной области
|   |   |-- classes/        # Классы предметной области
|   |   |   |-- concept_set.scs
|   |   |   |-- concept_oriented_set.scs
|   |   |-- relations/      # Отношения предметной области
|   |   |   |-- nrel_set_power.scs
|   |-- entities/          # Сущности предметной области
|   |   |-- set_A.scs
|   |-- actions/           # Классы действий, выполняемых
|   |                       агентами
|   |   |-- action_calculate_boolean.scs
```

Структурирование базы знаний на уровне её исходных файлов позволяет разделять знания по логическим блокам, что упрощает их разработку, тестирование и повторное использование.

4.2.2. Создание файла конфигурации базы знаний ostis-системы

Для исходных файлов базы знаний *ostis*-системы можно задать специальный файл конфигурации. Этот файл позволяет указать *sc-сборщику* базы знаний *ostis*-системы, какие директории необходимо включить в процесс сборки базы знаний.

Данным файлом конфигурации является *knowledge-base/repo.path*. Для него можно задать следующее содержимое:

```
subject-domain-of-sets/
```

Возможности файла конфигурации базы знаний *ostis*-системы описаны в разделах, описывающих разработку *ostis*-системы на базе примера *ostis*-системы.

4.2.3. Создание исходного файла заданной предметной области и её формализация на SCs-коде

База знаний любой *ostis*-системы может быть представлена как иерархия предметных областей и соответствующих им онтологий.



Каждая предметная область является описанием (спецификацией) понятий, экземпляров понятий и связей между экземплярами этих понятий.

Каждая онтология предметной области является описанием (спецификацией) понятий данной предметной области, связей между ними, а также свойств этих понятий.

В исходном файле *knowledge-base/subject-domain-of-sets/subject_domain_of_sets.scs* содержится формализация Предметной области множеств:

```
subject_domain_of_sets
<- concept_subject_domain;
=> nrel_main_idtf:
    [Предметная область множеств]
    (*
        <- lang_ru;;
    *);
[Subject domain of sets]
(*
    <- lang_en;;
*);
<= nrel_private_subject_domain:
    subject_domain_of_math_objects;
=> nrel_private_subject_domain:
    subject_domain_of_relations;
    subject_domain_of_structures;
-> rrel_maximum_studied_object_class:
    concept_set;
-> rrel_not_maximum_studied_object_class:
    concept_set_of_sets;
-> rrel_explored_relation:
    nrel_boolean;
=> nrel_note:
    [Данная предметная область описывает различные виды множеств.]
    (*
        <- lang_ru;;
    *);
[This subject domain describes different types of sets.]
```

```
(*
    <- lang_en;;
*);;
```

4.2.4. Создание исходных файлов абсолютных понятий и их формализация на SCs-коде

Любой понятие представляет собой класс сущностей (множество обладающих явно указанными общими свойствами), который является исследуемой сущностью хотя бы одной предметной области. Сущности могут быть представлять собой связи (связки) между другими сущностями. Понятия, которые представляют собой классы связей, называются относительными. Понятия, которые представляют собой классы сущностей, не являющихся связками, называются абсолютными.

Для заданной предметной области *subject_domain_of_sets* можно формализовать, например, 2 абсолютных понятия: *concept_set* и *concept_set_of_sets*.

Исходным файлом для формализации абсолютного понятия *concept_set* является файл *knowledge-base/subject-domain-of-sets/concepts/classes/concept_set.scs*. В нём может быть описана следующая информация:

```
concept_set
<- sc_node_class;
<- concept_class;
=> nrel_main_idtf:
    [множество]
    (*
        <- lang_ru;;
    *) ;
    [set]
    (*
        <- lang_en;;
    *) ;
=> nrel_idtf:
    [множество сущностей]
    (*
        <- lang_ru;;
    *) ;
```



```

    [set of entities]
    (*
        <- lang_en;;
    *);
<= nrel_inclusion:
    concept_math_object;
=> nrel_inclusion:
    concept_set_of_sets;;

```

Исходным файлом для формализации абсолютного понятия *concept_set_of_sets* является файл *knowledge-base/subject-domain-of-sets/concepts/classes/concept_set_of_sets.scs*. В нём может быть описана следующая информация:

```

concept_set_of_sets
<- sc_node_class;
<- concept_class;
=> nrel_main_idtf:
    [множество множеств]
    (*
        <- lang_ru;;
    *);
    [set of sets]
    (*
        <- lang_en;;
    *);
=> nrel_idtf:
    [семейство множества]
    (*
        <- lang_ru;;
    *);
    [family of sets]
    (*
        <- lang_en;;
    *);
<= nrel_inclusion:
    concept_set;;

```

При необходимости можно формализовать другие абсолютные понятия для заданной предметной области.

4.2.5. Создание исходных файлов относительных понятий и их формализация на SCs-коде

Относительные понятия представляют собой отношения между другими сущностями. Отношения могут быть ролевыми, то есть задавать роли для связей между множествами и их элементами, и, соответственно, могут быть неролевыми, то есть задавать неролевые связи между сущностями.

Для заданной предметной области *subject_domain_of_sets* можно формализовать, например, относительное понятие *nrel_boolean*.

Исходным файлом для формализации относительного понятия *nrel_boolean* является файл *knowledge-base/subject-domain-of-sets/concepts/relations/nrel_boolean.scs*. В нём может быть описана следующая информация:

```
nrel_boolean
<- sc_node_non_role_relation;
<- concept_non_role_relation;
<- concept_binary_relation;
<- concept_oriented_relation;
=> nrel_main_idtf:
    [булеан*]
    (*
        <- lang_ru;;
    *);
    [boolean*]
    (*
        <- lang_en;;
    *);
=> nrel_first_domain:
    concept_set;
=> nrel_first_domain:
    concept_set_of_sets;;
```

При необходимости можно формализовать другие относительные понятия для заданной предметной области.

4.2.6. Создание исходных файлов экземпляров понятий и их формализация на SCs-коде

Экземпляром понятия является сущность, которая принадлежит данному понятию.

Для заданной предметной области *subject_domain_of_sets* можно формализовать несколько экземпляров понятий. В качестве примера формализуем некоторое множество и булеан этого множества.

Исходным файлом для формализации данного множества является файл *knowledge-base/subject-domain-of-sets/entities/set_A.scs*. В нём может быть описана следующая информация:

```
set_A
=> nrel_main_idtf:
    [множество A]
    (*
        <- lang_ru;;
    *);
    [set A]
    (*
        <- lang_en;;
    *);
<- concept_set;
-> a;
-> b;
-> c;
=> nrel_boolean: {
    ... (* <- concept_empty_set;; *);
    {a} (* <- concept_singleton;; *);
    {b} (* <- concept_singleton;; *);
    {c} (* <- concept_singleton;; *);
    {a; b} (* <- concept_pair;; *);
    {a; c} (* <- concept_pair;; *);
    {b; c} (* <- concept_pair;; *);
    {a; b; c} (* <- concept_triple;; *)
} (* <- concept_set;; *);;
```

При необходимости можно формализовать другие экземпляры понятий для заданной предметной области.

4.2.7. Сборка базы знаний ostis-системы

Для сборки базы знаний следует использовать описанный ранее `sc-сборщик` базы знаний `ostis-системы` — `sc-builder` — который транслирует исходные файлы `.scs` и `.gwf` в бинарный формат `kb.bin`:

```
./sc-machine/bin/sc-builder \  
-i ./knowledge-base/repo.path \  
-o ./kb.bin \  
--clear
```

где:

- `-i` — путь к входной директории с исходными файлами базы знаний `ostis-системы`;
- `-o` — путь к выходной директории с бинарными файлами базы знаний `ostis-системы`;
- `--clear` — файл, указывающий на очистку выходной директории перед сборкой базы знаний `ostis-системы`.

Конечный результат сборки базы знаний `ostis-системы` должен выглядеть следующим образом:

```

[19:17:05][Info]: [sc-memory] Successfully initialized
[19:17:05][Info]: Initialize ScKeynodes
Parse repo path file...
Collect all sources...
Build knowledge base from sources...
[1/6]: ./knowledge-base/subject-domain-of-sets/actions/action_calculate_boolean.scs - ok
[2/6]: ./knowledge-base/subject-domain-of-sets/entities/set_A.scs - ok
[3/6]: ./knowledge-base/subject-domain-of-sets/concepts/classes/concept_set.scs - ok
[4/6]: ./knowledge-base/subject-domain-of-sets/concepts/classes/concept_set_of_sets.scs - ok
[5/6]: ./knowledge-base/subject-domain-of-sets/concepts/relations/nrel_boolean.scs - ok
[6/6]: ./knowledge-base/subject-domain-of-sets/subject_domain_of_sets.scs - ok
Clean state...
Statistics
Nodes: 275(33.293%)
Connectors: 410(49.6368%)
Links: 141(17.0702%)
Total: 826
[19:17:05][Info]: Shutdown ScKeynodes
[19:17:05][Info]: [sc-memory] Shutdown
[19:17:05][Info]: [sc-memory] Shutdown extensions
[19:17:05][Info]: [sc-memory] Extensions shutdown
[19:17:05][Info]: [sc-memory] Shutdown sc-helper
[19:17:05][Info]: [sc-fs-memory] Save sc-memory segments
[19:17:05][Info]:         Loaded segments count: 1
[19:17:05][Info]:         Sc-segments size: 4194312
[19:17:05][Info]:         Last not engaged segment num: 0
[19:17:05][Info]:         Last released segment num: 0
[19:17:05][Info]: [sc-fs-memory] Sc-memory segments saved
[19:17:05][Info]: [sc-fs-memory] Save sc-fs-memory dictionaries
[19:17:05][Info]: [sc-fs-memory] Dictionary `term - offsets` written
[19:17:05][Info]: [sc-fs-memory] Dictionary `string offsets - link hashes` written
[19:17:05][Info]:         Last string offset: 3992
[19:17:05][Info]: [sc-fs-memory] All sc-fs-memory dictionaries saved
[19:17:05][Info]: [sc-fs-memory] Shutdown
[19:17:05][Info]: [sc-fs-memory] Successfully shutdown
[19:17:05][Info]: [sc-memory] Shutdown context manager
[19:17:05][Info]: [sc-memory] Shutdown

```

4.3. Разработка решателя задач ostis-системы

Решатель задач представляет собой многоагентную систему, в которой каждый агент является автономным программным модулем, специализирующимся на решении определенного класса задач. Агенты реагируют на события в семантической памяти (sc-памяти) и выполняют соответствующие действия для обработки знаний.

4.3.1. Создание структуры модуля решателя задач ostis-системы

Под разработкой решателя задач ostis-системы прежде всего понимается разработка исходных файлов агентов этого решателя задач. Все исходные файлы решателя задач ostis-системы обычно разрабатываются в директории *problem-solver/*.

В данной директории реализуются платформенно-зависимые агенты на языке программирования C++. Агенты на C++ (C++ агенты) логически объединяются в модули.

В качестве примера рассмотрим процесс создания агент подсчёта булеана множества. Структура исходных файлов модуля решателя задач может выглядеть следующим образом:

```
problem-solver/
|-- CMakeLists.txt           # Конфигурация сборки решателя
задач
|-- set-processing-module/   # Директория модуля агентов
|-- CMakeLists.txt           # Конфигурация сборки модуля
|-- set_processing_module.hpp # Заголовочный файл модуля
|-- set_processing_module.cpp # Файл реализации модуля
|-- keynodes/                # Директория для ключевых узлов
|-- set_keynodes.hpp          # Заголовочный файл ключевых
узлов
|-- agent/                   # Директория для агентов
|-- calculate_boolean_agent.hpp # Заголовочный файл агента
|-- calculate_boolean_agent.cpp # Файл реализации агента
|-- test/                    # Директория для тестов
|-- calculate_boolean_agent_tests.cpp # Файл тестов
агента
```

Основной принцип разработки решателя задач — декомпозиция сложных задач на атомарные логические действия, что обеспечивает совместимость, модифицируемость и возможность повторного использования компонентов. Каждый агент инкапсулирует логику решения конкретной задачи и может работать независимо от других агентов, что делает систему устойчивой к отказам отдельных компонентов.

4.3.2. Создание исходного файла класса действий, выполняемых агентом, и его формализация на SCs-коде

Перед реализацией агентов следует определить классы действий, которые эти агенты будут выполнять.

Содержимое исходного файла *knowledge-base/subject-domain-of-sets/actions/action_calculate_boolean.scs* может выглядеть следующим образом:

```
action_calculate_boolean
<- sc_node_class;
<- concept_class;
```

```

=> nrel_main_idtf:
    [действие подсчёта булеана множества]
    (*
        <- lang_ru;;
    *);
    [action to calculate boolean of set]
    (*
        <- lang_en;;
    *);
<= nrel_inclusion:
    concept_information_action;;

```

4.3.3. Конфигурация CMake для решателя задач ostis-системы

Корневой файл *CMakeLists.txt* решателя задач координирует сборку всех его модулей. Он использует директиву *add_subdirectory()* для подключения подпроектов, каждый из которых имеет собственную конфигурацию сборки.

Файл *problem-solver/CMakeLists.txt* может выглядеть следующим образом:

```

# Определение подпроектов решателя задач
add_subdirectory(${CMAKE_CURRENT_SOURCE_DIR}/set-processing-module)

```

Директива *add_subdirectory()* указывает *CMake* обработать файл *CMakeLists.txt* в указанной директории. Переменная *CMAKE_CURRENT_SOURCE_DIR* содержит путь к директории текущего обрабатываемого файла *CMakeLists.txt*. По мере добавления новых модулей в решатель задач в этот файл следует добавлять соответствующие строки *add_subdirectory()* для каждого нового модуля.

4.3.4. Конфигурация CMake для модуля агентов

Файл *CMakeLists.txt* модуля определяет правила сборки для всех исходных файлов модуля, линковку с библиотеками и настройку тестирования.

Файл *problem-solver/set-processing-module/CMakeLists.txt* может выглядеть следующим образом:

```

# Находим все файлы с расширениями .cpp и .hpp в текущей
директории,
# поддиректориях agent и keynodes.
file(GLOB SOURCES CONFIGURE_DEPENDS

```

```

    "*.cpp" "*.hpp"
    "agent/*.cpp" "agent/*.hpp"
    "keynodes/*.hpp"
)

# Создаём динамическую библиотеку set-processing-module.
add_library(set-processing-module SHARED ${SOURCES})
# Добавляем открытую зависимость от библиотеки sc-memory из пакета
# sc-machine.
target_link_libraries(set-processing-module
    LINK_PUBLIC sc-machine::sc-memory
)
# Добавляем текущую директорию исходного кода в список путей для
# поиска заголовочных файлов.
target_include_directories(set-processing-module
    PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}
)
# Устанавливаем свойства для цели set-processing-module
set_target_properties(set-processing-module
    # Указываем директорию для вывода скомпилированной библиотеки.
    PROPERTIES LIBRARY_OUTPUT_DIRECTORY ${SC_EXTENSIONS_DIRECTORY}
)

# Собираем все файлы с расширением .cpp из директории test.
file(GLOB TEST_SOURCES CONFIGURE_DEPENDS
    "test/*.cpp"
)

# Создаём исполняемый файл для тестов.
add_executable(set-processing-module-tests ${TEST_SOURCES})
# Связываем тесты с библиотекой модуля с агентом.
target_link_libraries(set-processing-module-tests
    LINK_PRIVATE GTest::gtest_main
    LINK_PRIVATE set-processing-module
)
# Добавляем директорию с исходниками в список путей для поиска
# заголовочных файлов.

```



```

target_include_directories(set-processing-module-tests
    PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}
)

# Добавляем тесты в проект.
# TEST_LIST содержит список всех файлов с тестами.
# WORKING_DIRECTORY устанавливает рабочую директорию для запуска
# тестов.
gtest_discover_tests(set-processing-module-tests
    TEST_LIST ${TEST_SOURCES}
    WORKING_DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR}/test
)

```

Команда `file(GLOB ...)` автоматически находит все файлы, соответствующие указанным шаблонам. Опция `CONFIGURE_DEPENDS` (доступна начиная с *CMake* 3.12) указывает *CMake* перезапускать конфигурацию при добавлении или удалении файлов, соответствующих шаблонам. Это решает основную проблему глобирования — необходимость вручную перезапускать *CMake* при изменении списка исходных файлов.

Команда `add_library(... SHARED ...)` создаёт динамическую (разделяемую) библиотеку. Динамические библиотеки загружаются во время выполнения программы, что позволяет платформе загружать модули агентов по требованию.

Команда `target_link_libraries()` указывает, с какими библиотеками следует связать создаваемую цель. Опция `LINK_PUBLIC` означает, что зависимость от `sc-machine::sc-memory` будет транзитивной — все, кто использует наш модуль, автоматически получат доступ к `sc-памяти`. Библиотека `sc-memory` предоставляет API для работы с семантической памятью.

Команда `target_include_directories()` добавляет директории для поиска заголовочных файлов. Опция `PRIVATE` означает, что эти директории используются только при компиляции самого модуля, но не передаются зависимым целям.

Команда `set_target_properties()` устанавливает различные свойства для цели сборки. Свойство `LIBRARY_OUTPUT_DIRECTORY` определяет, куда будет помещена скомпилированная библиотека. Переменная `SC_EXTENSIONS_DIRECTORY` указывает на специальную директорию для

расширений *sc-machine*, откуда платформа загружает модули агентов во время выполнения.

Команда *add_executable()* создаёт исполняемый файл для запуска тестов. Библиотека *GTest::gtest_main* предоставляет готовую функцию *main()*, которая инициализирует фреймворк *Google Test* и запускает все зарегистрированные тесты.

Команда *gtest_discover_tests()* автоматически обнаруживает все тесты в исполняемом файле и регистрирует их в системе тестирования *CMake*. Это позволяет запускать тесты индивидуально и получать детальные отчёты о результатах тестирования.

4.3.5. Объявление ключевых *sc*-элементов для C++ агентов

ключевые sc-элементы — это *sc*-элементы, которые постоянно и многократно используются агентами.

Вместо того чтобы каждый раз искать эти элементы по их системным *sc*-идентификаторам, платформа предоставляет механизм кэширования их адресов для быстрого доступа.

Файл с ключевыми *sc*-элементами *problem-solver/set-processing-module/keynodes/set_processing_keynodes.hpp* может выглядеть следующим образом:

```
#include <sc-memory/sc_keynodes.hpp>

// Создаём класс, объединяющий ключевые sc-элементы, используемые
// агентами в рамках одного модуля.
// Наследуем его от базового класса ScKeynodes.
class SetProcessingKeynodes : public ScKeynodes
{
public:
    static inline ScKeynode const action_calculate_boolean{
        "action_calculate_boolean", ScType::ConstNodeClass};
    static inline ScKeynode const nrel_boolean{
        "nrel_boolean", ScType::ConstNodeNonRole};
    // Здесь первым аргументом конструктора является системный
    // sc-идентификатор ключевого sc-элемента, а второй аргумент —
    // тип этого sc-элемента.
```

```

// Если в sc-памяти нет ключевого sc-элемента с таким системным
// sc-идентификатором, то в ней будет создан sc-элемент с таким
// системным sc-идентификатором и указанным типом.
};

```

Класс *ScKeynodes* — это базовый класс платформы для работы с ключевыми узлами. Он обеспечивает инфраструктуру для кэширования и быстрого доступа к часто используемым sc-элементам.

Ключевое слово `static inline` (C++17) позволяет определять статические члены класса непосредственно в заголовочном файле без необходимости отдельного определения в `.cpp` файле. Это упрощает код и устраняет необходимость в дополнительных файлах реализации для простых констант.

Класс *ScKeynode* инкапсулирует системный sc-идентификатор и sc-адрес ключевого sc-элемента. При первом обращении к ключевому sc-узлу платформа ищет его в sc-памяти по системному sc-идентификатору и кэширует его sc-адрес. Если sc-элемент не найден, он создаётся с указанным типом. Это гарантирует, что все необходимые sc-элементы существуют в базе знаний.

Тип *ScType::ConstNodeClass* указывает, что *action_calculate_boolean* является константным sc-узлом, обозначающим sc-класс. Тип *ScType::ConstNodeNonRole* указывает, что *nrel_set_power* является константным sc-узлом, обозначающим неролевое отношение.

4.3.6. Реализация C++ агента

Агент — это автономный программный модуль, который реагирует на определённые события в sc-памяти и выполняет соответствующее действие в ней. Агенты могут реагировать на инициирование действий определённого класса.

Файл объявления класса агента *problem-solver/set-processing-module/agent/calculate_boolean_agent.hpp* может содержать следующее:

```

#pragma once
// Подключаем заголовочный файл для работы с агентами.
#include <sc-memory/sc_agent.hpp>
// Создаём класс, реализующий агент вычисления булеана множества.
// Наследуем его от класса ScActionInitiatedAgent, описывающего
// класс агентов, реагирующих на инициированное действие.

```

```

class CalculateBooleanAgent : public ScActionInitiatedAgent
{
public:
    // Объявляем метод, возвращающий sc-адрес класса действий,
    // выполняемых указанным агентом.
    ScAddr GetActionClass() const override;
    // Объявляем метод, реализующий основную логику указанного агента
    // - выполняющий инициированное действие и возвращающий результат
    // выполнения действия.
    ScResult DoProgram(ScAction & action) override;
};

```

Класс *ScActionInitiatedAgent* — это базовый класс платформы для агентов, реагирующих на события инициирования действий. Агенты этого типа автоматически подписываются на события создания действий определённого класса в множество инициированных действий.

Метод *GetActionClass()* возвращает sc-адрес класса действий, на который реагирует агент. Платформа использует эту информацию для определения, какой агент следует активировать при инициировании конкретного действия.

Метод *DoProgram()* содержит основную логику агента — алгоритм решения задачи. Он принимает объект действия (*ScAction*), выполняет его и возвращает результат выполнения.

Ключевое слово `override` (C++11) явно указывает, что метод переопределяет виртуальный метод базового класса. Это помогает компилятору обнаруживать ошибки, если сигнатура метода не совпадает с базовым классом.

Файл реализации агента *problem-solver/set-processing-module/agent/calculate_boolean_agent.cpp* может содержать следующее:

```

#include "calculate_boolean_agent.hpp"

#include <cmath>

// Подключаем все необходимые заголовочные файлы для работы с
// sc-памятью.
#include <sc-memory/sc_memory.hpp>

```

```

#include "keynodes/set_processing_keynodes.hpp"

ScAddr CalculateBooleanAgent::GetActionClass() const
{
    return SetProcessingKeynodes::action_calculate_boolean;
}

ScResult CalculateBooleanAgent::DoProgram(ScAction & action)
{
    // Получаем аргумент действия – sc-множество, для которого
    // необходимо посчитать булеан.
    auto const & [setAddr] = action.GetArguments<1>();

    // Если аргумент для действия не задан, то завершаем выполнение
    // действия с ошибкой.
    if (!m_context.IsElement(setAddr))
    {
        m_logger.Error("Set not specified.");
        return action.FinishWithError();
    }

    m_logger.Debug("Starting to calculate boolean for set");

    // Собираем все элементы исходного множества в вектор
    ScAddrVector elements;
    ScIterator3Ptr const it3 = m_context.CreateIterator3(
        setAddr,
        ScType::ConstPermPosArc,
        ScType::Unknown
    );
    while (it3->Next())
    {
        ScAddr const & elementAddr = it3->Get(2);
        elements.push_back(elementAddr);
    }

    m_logger.Debug("Found ", elements.size(), " elements in set");
}

```

```

// Вычисляем количество подмножеств:  $2^n$ 
size_t const elementCount = elements.size();
size_t const subsetsCount = std::pow(2, elementCount);

// Создаём sc-структуру для булеана множества
ScAddr const booleanAddr =
    m_context.GenerateNode(ScType::ConstNode);

// Создаём все подмножества
for (size_t mask = 0; mask < subsetsCount; ++mask)
{
    // Создаём sc-узел для текущего подмножества
    ScAddr const subsetAddr =
        m_context.GenerateNode(ScType::ConstNode);

    // Добавляем текущее подмножество в булеан
    m_context.GenerateConnector(
        ScType::ConstPermPosArc,
        booleanAddr,
        subsetAddr
    );

    // Заполняем подмножество элементами согласно маске
    for (size_t i = 0; i < elementCount; ++i)
    {
        // Проверяем, установлен ли i-й бит в маске
        if (mask & (1 << i))
        {
            // Если бит установлен, добавляем элемент в подмножество
            m_context.GenerateConnector(
                ScType::ConstPermPosArc,
                subsetAddr,
                elements[i]
            );
        }
    }
}

```

```

}

m_logger.Debug(
    "Generated ", subsetsCount, " subsets");

// Устанавливаем связь между исходным множеством и его булеаном
ScAddr const arcAddr = m_context.GenerateConnector(
    ScType::ConstCommonArc,
    setAddr,
    booleanAddr
);

ScAddr const booleanRelationArcAddr =
m_context.GenerateConnector(
    ScType::ConstPermPosArc,
    SetProcessingKeynodes::nrel_boolean,
    arcAddr
);

// Создаём структуру результата
ScStructure resultStructure = m_context.GenerateStructure();

// Добавляем булеан в структуру результата
resultStructure << booleanAddr;

// Добавляем все подмножества в структуру результата
ScIterator3Ptr const resultIt3 = m_context.CreateIterator3(
    booleanAddr,
    ScType::ConstPermPosArc,
    ScType::ConstNode
);
while (resultIt3->Next())
{
    ScAddr const & subsetAddr = resultIt3->Get(2);
    resultStructure << resultIt3->Get(1) << subsetAddr;

    // Добавляем элементы подмножества

```

```

    ScIterator3Ptr const subsetIt3 = m_context.CreateIterator3(
        subsetAddr,
        ScType::ConstPermPosArc,
        ScType::Unknown
    );

    while (subsetIt3->Next())
    {
        resultStructure << subsetIt3->Get(1) << subsetIt3->Get(2);
    }
}

// Добавляем отношение nrel_boolean в структуру результата
resultStructure << arcAddr
                << booleanRelationArcAddr
                << SetProcessingKeynodes::nrel_boolean;

// Устанавливаем структуру результата для действия
action.SetResult(resultStructure);

m_logger.Info("Boolean calculated successfully");

// Завершаем выполнение действия успешно
return action.FinishSuccessfully();
}

```

Метод `action.GetArguments<N>()` извлекает аргументы действия. Шаблонный параметр `<1>` указывает, что ожидается ровно один аргумент. Использование структурированных привязок (structured bindings, C++17) `auto const & [setAddr]` позволяет элегантно распаковать результат в переменную.

Поле `m_context` — это контекст работы с sc-памятью, предоставляемый базовым классом агентов. Он предоставляет методы для создания, поиска и модификации sc-элементов.

Метод `m_context.IsElement()` проверяет, является ли переданный sc-адрес корректным, то есть есть ли для него sc-элемент в sc-памяти.

Член *m_logger* — это объект для логирования, предоставляемый базовым классом. Он поддерживает различные уровни логирования: *Debug* (отладочная информация), *Info* (информационные сообщения), *Error* (ошибки).

Метод *action.FinishWithError()* завершает выполнение действия с ошибкой. Это переводит действие в состояние "завершено с ошибкой" и уведомляет систему о неуспешном выполнении.

Класс *ScIterator3* представляет итератор для поиска sc-конструкций из трёх элементов: начальный элемент, sc-дуга, конечный элемент. Это основной инструмент для навигации по семантической сети.

Метод *m_context.CreateIterator3()* создаёт итератор по заданному шаблону. В данном случае ищутся все sc-элементы, в которые входят константные sc-дуги постоянной положительной принадлежности из *setAddr*. Тип *ScType::Unknown* означает, что тип конечного элемента может быть любым.

Тип *ScType::ConstPermPosArc* обозначает все константные sc-дуги постоянной положительной принадлежности. Тип *ScType::ConstCommonArc* обозначает все константные sc-дуги общего вида.

Метод *m_context.GenerateNode()* создаёт новый sc-узел в sc-памяти. Метод *m_context.GenerateConnector()* создаёт новую sc-дугу, соединяющую два sc-элемента.

4.3.7. Создание модуля и регистрация C++ агента

Для того чтобы платформа могла загрузить и использовать модуль агентов, следует зарегистрировать модуль и объявить, какие агенты он содержит. Это процесс связывания динамической библиотеки модуля с платформой и подписки агентов на соответствующие события.

Файл объявления класса модуля *problem-solver/set-processing-module/set_processing_module.hpp* может быть задан следующим образом:

```
#pragma once

#include <sc-memory/sc_module.hpp>

// Создаём класс модуля и наследуем его от базового класса
ScModule.
class SetProcessingModule : public ScModule
```

```
{
    // Тут не нужно реализовывать никаких методов.
};
```

Класс *ScModule* — это базовый класс платформы для всех модулей расширений. Он предоставляет инфраструктуру для инициализации и завершения работы модуля, а также для управления жизненным циклом агентов.

Минимальная реализация модуля не требует переопределения каких-либо методов базового класса. Однако при необходимости можно переопределить методы *Initialize()* и *Shutdown()* для выполнения специфической инициализации и очистки ресурсов модуля.

Файл реализации класса модуля *problem-solver/set-processing-module/set_processing_module.cpp* может быть задан следующим образом:

```
#include "set_processing_module.hpp"

#include "agent/calculate_boolean_agent.hpp"

SC_MODULE_REGISTER(SetProcessingModule)
{
    ->Agent<CalculateBooleanAgent>();
    // Этот метод указывает модулю, что агент класса
    // CalculateBooleanAgent должен быть подписан на sc-событие
    // добавления выходящей sc-дуги из sc-элемента action_initiated.

    // Такой способ подписки агентов упрощает написание кода.
    // Вам не нужно думать об отписке агентов после
    // выключения системы — ваш модуль сделает это сам.
```

Макрос *SC_MODULE_REGISTER()* регистрирует модуль в платформе. Он создаёт необходимые точки входа для динамической загрузки библиотеки и связывает класс модуля с платформой.

Метод *Agent<AgentClass>()* регистрирует агент указанного класса в модуле. Платформа автоматически создаёт экземпляр агента, вызывает его метод *GetActionClass()* для определения класса действий и подписывает агент на события инициирования действий этого класса.

Подписка происходит через событийно-ориентированный механизм платформы. Когда в систему поступает запрос на выполнение действия, создаётся экземпляр действия в sc-памяти, который добавляется в множество инициированных действий (*action_initiated*). Это генерирует sc-событие добавления sc-дуги принадлежности, на которое подписаны соответствующие агенты. Агент, класс действий которого совпадает с классом инициированного действия, активируется и выполняет метод *DoProgram()*.

Преимущество такого декларативного подхода к регистрации — автоматическое управление жизненным циклом агентов. Разработчику не нужно вручную подписывать и отписывать агентов — платформа делает это автоматически при загрузке и выгрузке модуля. Это предотвращает утечки ресурсов и упрощает код.

4.3.8. Тестирование C++ агента

Тестирование агентов важно для обеспечения корректности их работы и предотвращения регрессий. *Google Test (gtest)* — это популярный фреймворк для модульного тестирования C++ кода, который обеспечивает богатый набор утверждений (assertions) и удобную организацию тестов.

Реализация тестов для разработанного C++ агента в исходном файле *problem-solver/set-processing-module/test/calculate_boolean_agent_tests.cpp* может выглядеть следующим образом:

```
// Подключаем заголовочный файл для тестирования агентов
#include <sc-memory/test/sc_test.hpp>

#include <sc-memory/sc_memory.hpp>

#include "agent/calculate_boolean_agent.hpp"
#include "keynodes/set_processing_keynodes.hpp"

using AgentTest = ScMemoryTest;

TEST_F(AgentTest, CalculateBooleanAgentFinishedSuccessfully)
{
    // Регистрируем агент в sc-памяти
```

```
m_ctx->SubscribeAgent<CalculateBooleanAgent>();

// Создаём действие с классом для выполнения агентом
ScAction action = m_ctx->GenerateAction(
    SetProcessingKeynodes::action_calculate_boolean);

// Создаём множество с двумя sc-элементами
ScAddr const setAddr = m_ctx->GenerateNode(ScType::ConstNode);
ScAddr const nodeAddr1 = m_ctx->GenerateNode(ScType::ConstNode);
ScAddr const nodeAddr2 = m_ctx->GenerateNode(ScType::ConstNode);

m_ctx->GenerateConnector(
    ScType::ConstPermPosArc,
    setAddr,
    nodeAddr1);
m_ctx->GenerateConnector(
    ScType::ConstPermPosArc,
    setAddr,
    nodeAddr2);

// Устанавливаем созданное множество как аргумент для действия
action.SetArguments(setAddr);

// Иницилируем и ждём, пока действие будет завершено
EXPECT_TRUE(action.InitiateAndWait());

// Проверяем, что действие завершилось успешно
EXPECT_TRUE(action.IsFinishedSuccessfully());

// Получаем структуру результата действия
ScStructure const resultStructure = action.GetResult();

// Проверяем, что она содержит sc-элементы
EXPECT_FALSE(resultStructure.IsEmpty());

// Проверяем, что в результате есть узел булеана
ScAddr booleanAddr;
```

```

ScIterator5Ptr it5 = m_ctx->CreateIterator5(
    setAddr,
    ScType::ConstCommonArc,
    ScType::ConstNode,
    ScType::ConstPermPosArc,
    SetProcessingKeynodes::nrel_boolean
);

EXPECT_TRUE(it5->Next());
booleanAddr = it5->Get(2);
EXPECT_TRUE(booleanAddr.IsValid());

// Проверяем, что булеан содержит правильное количество
// подмножеств
// Для множества из 2 элементов булеан содержит  $2^2 = 4$ 
// подмножества
size_t subsetsCount = 0;
ScIterator3Ptr it3 = m_ctx->CreateIterator3(
    booleanAddr,
    ScType::ConstPermPosArc,
    ScType::ConstNode
);

while (it3->Next())
{
    subsetsCount++;
}

EXPECT_EQ(subsetsCount, 4u);

// Проверяем наличие пустого подмножества
bool hasEmptySubset = false;
it3 = m_ctx->CreateIterator3(
    booleanAddr,
    ScType::ConstPermPosArc,
    ScType::ConstNode
);

```

```

while (it3->Next())
{
    ScAddr const subsetAddr = it3->Get(2);

    // Подсчитываем элементы в подмножестве
    size_t elementCount = 0;
    ScIterator3Ptr subsetIt3 = m_ctx->CreateIterator3(
        subsetAddr,
        ScType::ConstPermPosArc,
        ScType::Unknown
    );

    while (subsetIt3->Next())
    {
        elementCount++;
    }

    // Пустое подмножество не содержит элементов
    if (elementCount == 0)
    {
        hasEmptySubset = true;
    }
}

EXPECT_TRUE(hasEmptySubset);

// Проверяем наличие полного подмножества (самого множества)
bool hasFullSubset = false;
it3 = m_ctx->CreateIterator3(
    booleanAddr,
    ScType::ConstPermPosArc,
    ScType::ConstNode
);

while (it3->Next())
{

```

```

ScAddr const subsetAddr = it3->Get(2);

// Проверяем, содержит ли подмножество оба элемента
bool hasNode1 = m_ctx->CheckConnector(
    subsetAddr,
    nodeAddr1,
    ScType::ConstPermPosArc);
bool hasNode2 = m_ctx->CheckConnector(
    subsetAddr,
    nodeAddr2,
    ScType::ConstPermPosArc);

if (hasNode1 && hasNode2)
{
    hasFullSubset = true;
}
}

EXPECT_TRUE(hasFullSubset);

// Проверяем наличие одноэлементных подмножеств
size_t singleElementSubsets = 0;
it3 = m_ctx->CreateIterator3(
    booleanAddr,
    ScType::ConstPermPosArc,
    ScType::ConstNode
);

while (it3->Next())
{
    ScAddr const subsetAddr = it3->Get(2);

    // Подсчитываем элементы в подмножестве
    size_t elementCount = 0;
    ScIterator3Ptr subsetIt3 = m_ctx->CreateIterator3(
        subsetAddr,
        ScType::ConstPermPosArc,

```

```

        ScType::Unknown
    );

    while (subsetIt3->Next())
    {
        elementCount++;
    }

    if (elementCount == 1)
    {
        singleElementSubsets++;
    }
}

EXPECT_EQ(singleElementSubsets, 2u);

// Дeregистрируем агент в sc-памяти
m_ctx->UnsubscribeAgent<CalculateBooleanAgent>();
}

TEST_F(AgentTest, CalculateBooleanAgentEmptySet)
{
    // Регистрируем агент в sc-памяти
    m_ctx->SubscribeAgent<CalculateBooleanAgent>();

    // Создаём действие
    ScAction action = m_ctx->GenerateAction(
        SetProcessingKeynodes::action_calculate_boolean);

    // Создаём пустое множество
    ScAddr const emptySetAddr =
        m_ctx->GenerateNode(ScType::ConstNode);

    // Устанавливаем пустое множество как аргумент
    action.SetArguments(emptySetAddr);

    // Иницилируем и ждём завершения

```



```

EXPECT_TRUE(action.InitiateAndWait());

// Проверяем успешное завершение
EXPECT_TRUE(action.IsFinishedSuccessfully());

// Получаем булеан
ScAddr booleanAddr;
ScIterator5Ptr it5 = m_ctx->CreateIterator5(
    emptySetAddr,
    ScType::ConstCommonArc,
    ScType::ConstNode,
    ScType::ConstPermPosArc,
    SetProcessingKeynodes::nrel_boolean
);

EXPECT_TRUE(it5->Next());
booleanAddr = it5->Get(2);

// Булеан пустого множества содержит только одно подмножество —
// само пустое множество
size_t subsetsCount = 0;
ScIterator3Ptr it3 = m_ctx->CreateIterator3(
    booleanAddr,
    ScType::ConstPermPosArc,
    ScType::ConstNode
);

while (it3->Next())
{
    subsetsCount++;
}

EXPECT_EQ(subsetsCount, 1u);

// Дерегистрируем агент
m_ctx->UnsubscribeAgent<CalculateBooleanAgent>();
}

```

```
TEST_F(AgentTest, CalculateBooleanAgentInvalidArgument)
```

```
{  
    // Регистрируем агент в sc-памяти  
    m_ctx->SubscribeAgent<CalculateBooleanAgent>();  
  
    // Создаём действие  
    ScAction action = m_ctx->GenerateAction(  
        SetProcessingKeynodes::action_calculate_boolean);  
  
    // Иницилируем и ждём завершения  
    EXPECT_TRUE(action.InitiateAndWait());  
  
    // Проверяем, что действие завершилось с ошибкой  
    EXPECT_TRUE(action.IsFinishedWithError());  
  
    // Дерегистрируем агент  
    m_ctx->UnsubscribeAgent<CalculateBooleanAgent>();  
}
```

```
TEST_F(AgentTest, CalculateBooleanAgentThreeElements)
```

```
{  
    // Регистрируем агент  
    m_ctx->SubscribeAgent<CalculateBooleanAgent>();  
  
    // Создаём действие  
    ScAction action = m_ctx->GenerateAction(  
        SetProcessingKeynodes::action_calculate_boolean);  
  
    // Создаём множество с тремя элементами  
    ScAddr const setAddr = m_ctx->GenerateNode(ScType::ConstNode);  
    ScAddr const nodeAddr1 = m_ctx->GenerateNode(ScType::ConstNode);  
    ScAddr const nodeAddr2 = m_ctx->GenerateNode(ScType::ConstNode);  
    ScAddr const nodeAddr3 = m_ctx->GenerateNode(ScType::ConstNode);  
  
    m_ctx->GenerateConnector(  
        ScType::ConstPermPosArc, setAddr, nodeAddr1);  
}
```

```

m_ctx->GenerateConnector(
    ScType::ConstPermPosArc, setAddr, nodeAddr2);
m_ctx->GenerateConnector(
    ScType::ConstPermPosArc, setAddr, nodeAddr3);

// Устанавливаем аргумент
action.SetArguments(setAddr);

// Иницилируем и ждём
EXPECT_TRUE(action.InitiateAndWait());

// Проверяем успешное завершение
EXPECT_TRUE(action.IsFinishedSuccessfully());

// Получаем булеан
ScAddr booleanAddr;
ScIterator5Ptr it5 = m_ctx->CreateIterator5(
    setAddr,
    ScType::ConstCommonArc,
    ScType::ConstNode,
    ScType::ConstPermPosArc,
    SetProcessingKeynodes::nrel_boolean
);

EXPECT_TRUE(it5->Next());
booleanAddr = it5->Get(2);

// Для множества из 3 элементов булеан содержит  $2^3 = 8$ 
// подмножеств
size_t subsetsCount = 0;
ScIterator3Ptr it3 = m_ctx->CreateIterator3(
    booleanAddr,
    ScType::ConstPermPosArc,
    ScType::ConstNode
);

while (it3->Next())

```

```

{
    subsetsCount++;
}

EXPECT_EQ(subsetsCount, 8u);

// Дерегистрируем агент
m_ctx->UnsubscribeAgent<CalculateBooleanAgent>();
}

```

Класс *ScMemoryTest* — это базовый класс платформы для тестов, работающих с sc-памятью. Он автоматически создаёт и инициализирует временную sc-память перед каждым тестом и очищает её после завершения теста.

Макрос *TEST_F(TestFixture, TestName)* определяет тест, использующий фикстуру (test fixture). Фикстура — это класс, предоставляющий общее окружение для группы тестов. Первый параметр — имя класса фикстуры, второй — имя конкретного теста.

Поле *m_ctx* — это контекст работы с sc-памятью, предоставляемый фикстурой *ScMemoryTest*. Он доступен во всех тестах, использующих эту фикстуру.

Метод *m_ctx->SubscribeAgent<AgentClass>()* регистрирует агент в тестовой sc-памяти. Это необходимо, так как тесты работают в изолированной памяти, отдельной от основной системы.

Метод *m_ctx->GenerateAction()* создаёт экземпляр действия указанного класса. Это специализированный метод для удобной работы с действиями в тестах.

Метод *action.InitiateAndWait()* иницирует выполнение действия и ожидает его завершения в течение указанного времени. Это синхронный метод, который блокирует выполнение до завершения действия или истечения таймута.

Макрос *EXPECT_TRUE(condition)* — это нефатальное утверждение Google Test. Если условие ложно, тест помечается как провалившийся, но продолжает выполнение. Существует также фатальная версия *ASSERT_TRUE()*, которая прерывает выполнение теста при провале.

Макрос `EXPECT_FALSE(condition)` проверяет, что условие ложно. Макрос `EXPECT_EQ(expected, actual)` проверяет равенство двух значений.

Класс *ScIterator5* представляет итератор для поиска sc-конструкций из пяти sc-элементов: начальный элемент, первая дуга, промежуточный элемент, вторая дуга, конечный элемент. Это используется для поиска бинарных отношений: в данном случае ищется отношение *nrel_boolean* между множеством и его булеаном.

4.3.8. Сборка решателя задач ostis-системы

Для сборки решателя задач ostis-системы (в том числе разработанного агента и тестов для него) необходимо выполнить следующие команды:

```
# Конфигурирование проекта с использованием CMake preset
cmake --preset conan-release

# Сборка проекта
cmake --build --preset conan-release
```

Данные команды следует запускать после каждого изменения в исходных файлах решателя задач ostis-системы.

Конечный результат сборки решателя задач ostis-системы должен выглядеть следующим образом:

```
[ 40%] Building CXX object problem-solver/set-processing-module/CMakeFiles/set-processing-module.dir/agent/calculate_boolean_agent.cpp.o
[ 40%] Building CXX object problem-solver/set-processing-module/CMakeFiles/set-processing-module.dir/set_processing_module.cpp.o
[ 60%] Linking CXX shared library ../../lib/extensions/libset-processing-module.dylib
[ 60%] Built target set-processing-module
[ 80%] Building CXX object problem-solver/set-processing-module/CMakeFiles/set-processing-module-tests.dir/test/calculate_boolean_agent_tests.cpp.o
[100%] Linking CXX executable ../../bin/set-processing-module-tests
[100%] Built target set-processing-module-tests
```

Скомпилированные расширения (библиотеки агентов) будут размещены в директории *build/Release/extensions/*.

4.3.9. Запуск тестов C++ агента

После успешной сборки решателя задач ostis-системы можно запустить и проверить тесты, выполнив следующую команду:

```
./build/Release/bin/set-processing-module-tests
```

Конечный результат запуска тестов C++ агента должен выглядеть следующий образом:

```
[18:40:47][Info]: CalculateBooleanAgent: Agent `CalculateBooleanAgent` finished checking result condition.
[18:40:47][Info]: Unsubscribe agent implementation `CalculateBooleanAgent` from event `ScEventAfterGenerateOutgoingArc` with subscription sc-element `action_initiated`.
[18:40:47][Info]: Shutdown ScKeynodes
[ OK ] AgentTest.CalculateBooleanAgentThreeElements (8 ms)
[-----] 4 tests from AgentTest (38 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (38 ms total)
[ PASSED ] 4 tests.
```

4.3.10. Запуск ostis-системы

Также после успешной сборки базы знаний ostis-системы можно запустить sc-машину с собранной базой знаний и скомпилированными расширениями модулей агентов:

```
./sc-machine/bin/sc-machine \
-s kb.bin \
-e "sc-machine/lib/extensions;build/Release/lib/extensions"
```

где:

- -s — путь к директории с бинарными файлами базы знаний ostis-системы;
- -e — пути к директориям с расширениями (библиотеками агентов), разделённые точкой с запятой.

4.4. Использование стандартного пользовательского интерфейса для ostis-систем

Будет дополнено.

Полезные материалы

1. [Презентация о Технологии OSTIS](#)
2. [Документация sc-машины](#)
3. [Документация программного интерфейса sc-памяти](#)
4. [Статья по формальной модели sc-памяти](#)
5. [Гайд по разработке простого агента на C++](#)
6. [Презентация-гайд по разработке простого агента на C++](#)
7. [Статья по программному интерфейсу для разработки C++ агентов](#)
8. [Статья-руководство по разработке компонентов ostis-систем](#)

Контрольные вопросы

1. Дайте определение Технологии OSTIS и объясните её основное назначение в контексте разработки интеллектуальных систем.
2. Назовите четыре основных принципа Технологии OSTIS и кратко объясните каждый из них.
3. Объясните концепцию "семантической совместимости" в контексте OSTIS. Как это решает проблему совместимости в технологиях?
4. Какие три основных компонента входят в состав ostis-системы? Кратко опишите функцию каждого.
5. Объясните разницу между базой знаний и решателем задач в контексте ostis-системы.
6. Что такое пользовательский интерфейс ostis-системы с точки зрения её архитектуры?
7. Почему важно, что каждый компонент ostis-системы может быть полностью представлен на SC-коде?
8. Из каких основных компонентов состоит Программная платформа ostis-систем?
9. Объясните понятие "sc-памяти" и её роль в платформе.
10. Какие подязыки SC-кода вам известны? Приведите примеры их применения.
11. Назовите три основных репозитория Проекта OSTIS и объясните назначение каждого.
12. Какова основная задача Проекта OSTIS?
13. Почему рекомендуется использовать проект примера ostis-системы в качестве базы для разработки?
14. Назовите три ключевые директории проекта примера ostis-системы и их назначение.
15. Какие файлы исходников базы знаний вы знаете? Чем они отличаются?
16. Что такое фрагмент базы знаний и почему нужно разделять знания на фрагменты?
17. Объясните понятие "исходный файл базы знаний".
18. Что такое "сборка базы знаний" и для чего она нужна?
19. Объясните роль sc-сборщика в разработке ostis-системы.
20. Что такое "бинарный файл базы знаний" и чем он отличается от исходных файлов?
21. Какова цель тестирования формализованных фрагментов базы знаний?

22. Какие два сервера необходимо запустить для работы ostis-системы и в чем их различие?
23. Объясните понятие "sc-машина" и её функции.
24. Что такое "sc-веб" и как он взаимодействует со sc-машиной?
25. На каком localhost-адресе доступен веб-интерфейс ostis-системы после запуска?
26. Почему нельзя закрывать терминалы во время работы sc-машины и sc-веба?
27. Какая комбинация клавиш используется для остановки приложений?
28. Что включает в себя процесс инициализации проекта ostis-системы?
29. Назовите основные директории, которые нужно создать при инициализации нового проекта.
30. Объясните роль файла conanfile.txt в проекте.
31. Что такое Conan и для чего он используется в проекте ostis-системы?
32. Объясните назначение файла CMakeLists.txt.
33. Что такое CMake и почему используется кроссплатформенная система сборки?
34. Что такое cmake и какую роль он играет в процессе разработки?
35. Объясните различие между build-essential, ninja-build и другими инструментами.
36. Почему важна установка Python и pip?
37. Что такое pipx и чем он отличается от обычного pip?
38. Для чего нужно выполнить команду conan remote add ostis-ai?
39. Объясните назначение команды conan profile detect.
40. Что означает опция --build=missing в команде conan install?
41. Какие инструменты входят в состав sc-машины после установки?
42. Какова типичная структура директорий при разработке базы знаний?
43. Объясните разницу между абсолютными и относительными понятиями.
44. Что такое "экземпляр понятия"?
45. Назовите основные типы элементов, которые используются при формализации базы знаний на SCs-коде.
46. Объясните назначение => в SCs-коде.
47. Что обозначает <= в SCs-коде?
48. Объясните назначение -> в контексте выражения отношений.
49. Что означает тройка скобок (* ... *) в SCs-коде?
50. Какие идентификаторы используются для указания языка в базе знаний?
51. Что такое "абсолютное понятие"? Приведите пример из документа.

52. Объясните структуру формализации абсолютного понятия `concept_set`.
53. Что такое "относительное понятие" и в чем его отличие от абсолютного?
54. Объясните разницу между ролевыми и неролевыми отношениями.
55. Приведите пример формализации относительного понятия из документа.
56. Как в SCs-коде описывается принадлежность экземпляра к понятию?
57. Что такое "решатель задач" в контексте ostis-системы?
58. Объясните многоагентный подход в решателе задач.
59. Какие основные директории должны быть в структуре решателя задач?
60. Что такое "модуль решателя задач"?
61. Объясните принцип декомпозиции в разработке решателя задач.
62. Что такое "класс действий"?
63. Как формализуется класс действий на SCs-коде? Приведите пример.
64. Объясните связь между классом действий и типом агента, который его выполняет.
65. Объясните роль файла `CMakeLists.txt` в корне проекта.
66. Что такое директива `add_subdirectory()` и почему она используется?
67. Объясните различие между `add_library() SHARED` и `STATIC`.
68. Что означает опция `LINK_PUBLIC` в команде `target_link_libraries()`?
69. Объясните назначение переменной `SC_EXTENSIONS_DIRECTORY`.
70. Для чего используется команда `gtest_discover_tests()`?
71. Что такое "ключевые sc-элементы" и зачем они нужны?
72. Объясните назначение класса `ScKeynodes`.
73. Что означает `static inline` в объявлении ключевого узла?
74. Объясните параметры конструктора `ScKeynode`.
75. Что происходит, если ключевой sc-элемент не найден в sc-памяти?
76. Какие типы используются для обозначения классов и отношений?
77. Что такое "агент" в контексте ostis-системы?
78. Объясните различие между `ScActionInitiatedAgent` и другими типами агентов.
79. Что возвращает метод `GetActionClass()`?
80. Объясните сигнатуру метода `DoProgram()` и его значение.
81. Что такое `ScResult` в контексте выполнения действия?
82. Объясните назначение `m_context` в реализации агента.
83. Что такое `m_logger` и какие уровни логирования вам известны?
84. Объясните метод `m_context.CreateIterator3()` и его параметры.
85. Что означает тип `ScType::Unknown` в контексте итератора?

86. Объясните различие между `ScType::ConstPermPosArc` и `ScType::ConstCommonArc`.
87. Что такое `ScAddr` и какова его роль в работе с памятью?
88. Объясните назначение методов `GenerateNode()` и `GenerateConnector()`.
89. Что такое `ScStructure` и как она используется в результатах действий?
90. Какова роль класса `ScModule` в платформе?
91. Объясните назначение макроса `SC_MODULE_REGISTER()`.
92. Что делает метод `Agent<AgentClass>()`?
93. Объясните событийно-ориентированный механизм подписки агентов.
94. Почему декларативный подход к регистрации предпочтителен?
95. Что такое Google Test (gtest) и зачем он используется?
96. Объясните назначение класса `ScMemoryTest`.
97. Что такое "фикстура теста" и какова её роль?
98. Объясните макрос `TEST_F()` и его параметры.
99. Объясните этапы подготовки агента к тестированию.
100. Что делает метод `m_ctx->SubscribeAgent<AgentClass>()`?
101. Объясните разницу между `EXPECT_TRUE()` и `ASSERT_TRUE()`.
102. Для чего используется метод `action.InitiateAndWait()`?
103. Что проверяют методы `IsFinishedSuccessfully()` и `IsFinishedWithError()`?
104. Назовите четыре типа тестов для агента из примера документа.
105. Объясните цель теста `CalculateBooleanAgentFinishedSuccessfully`.
106. Почему важен тест `CalculateBooleanAgentInvalidArgument`?
107. Объясните тест для пустого множества и его значение.
108. Зачем тестируются граничные случаи (пустое множество, три элемента)?
109. Объясните назначение `ScIterator5` в тестах.
110. Что означает первый параметр в `CreateIterator5()`?
111. Объясните методы `Get(2)` и `Get(4)` в контексте итератора.
112. Что проверяет метод `IsValid()` для `sc`-адреса?

Индивидуальное задание

1. Выбрать вариант из перечня вариантов индивидуальных заданий.
2. Для заданного варианта выполнить задания, указанные в требованиях, таким образом, чтобы в результате получилась *ostis*-система, в рамках которой:
 - формализована предметная область базы знаний, заданная в варианте задания;

- разработан модуль решателя задач с агентами, выполняющими задачи, указанные в требованиях задачи.

3. Составить отчёт о проделанной работе.

Типовая структура отчёта

1. Титульный лист с указанием названия работы, ФИО студентов, группы, преподавателя.
2. Индивидуальное задание: формулировка задачи, описание предметной области.
3. Реализация базы знаний:
 - a. Описание разработанной предметной области.
 - b. Листинги основных файлов на SCs-коде.
 - c. Описание ключевых понятий, отношений, сущностей и действий.
4. Реализация агентов:
 - a. Описание логики работы агентов.
 - b. Листинги исходного кода агентов с комментариями.
 - c. Описание ключевых sc-элементов и вспомогательных методов.
5. Тестирование:
 - a. Описание тестовых сценариев.
 - b. Листинги тестов.
 - c. Результаты выполнения модульных тестов.
 - d. Скриншоты работы системы через веб-интерфейс.
6. Выводы о проделанной работе.
7. Список использованных источников.

Примечание к заданию

Задание по расчётной работе может выполняться в командах до 3 человек.

ВАРИАНТЫ ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ

Вариант 1. Анализ структуры управления компании.

Постановка:

В компании "ТехПром" действует строго иерархическая схема управления. Генеральный директор (Иванов А.С.) руководит тремя направлениями: коммерческим, IT и финансовым.

Иерархическая схема управления представляет собой следующее дерево:

- Генеральный директор
 - Коммерческий директор
 - Менеджер по продажам
 - Менеджер по закупкам
 - IT-директор
 - Системный администратор
 - Программист 1
 - Программист 2
 - Финансовый директор
 - Главный бухгалтер
 - Экономист

Входные данные:

Список всех должностей компании с явными связями «кто подчинён кому» (см. пример выше).

Требуется:

1. Построить граф управления (вершины — должности, рёбра — прямое подчинение).
2. Проверить выполнение свойств дерева: граф связан, не содержит циклов, число рёбер = $V-1$.
3. Найти корневую вершину (директор).
4. Вычислить глубину каждой позиции и высоту дерева.
5. Определить листовые должности (без подчинённых).

Вариант 2. Анализ корректности плана разработки программного обеспечения.

Постановка:

Компания запускает проект по созданию мобильного приложения "Доставка+". Разработка разбита на последовательные этапы: анализ требований, проектирование архитектуры, дизайн интерфейса, программирование backend и frontend, тестирование и развертывание сервиса. Для успешной реализации важно, чтобы проектный план не содержал логических ошибок, все зависимости между задачами были корректно отражены, а сроки не превышали минимально возможные.

Пример:

План включает следующие задачи:

1. Анализ требований (A) — 14 дней.
2. Проектирование архитектуры (B) — 21 день, зависит от A.
3. Дизайн интерфейса (C) — 14 дней, зависит от A.
4. Разработка backend (D) — 35 дней, зависит от B.
5. Разработка frontend (E) — 28 дней, зависит от B и C.
6. Тестирование (F) — 21 день, зависит от D и E.
7. Развертывание (G) — 7 дней, зависит от F.

Входные данные:

Список этапов проекта с их продолжительностью и зависимостями (см. пример выше).

Требуется:

1. Построить ориентированный ациклический граф (DAG) задач.
2. Проверить корректность плана: отсутствуют ли циклы, можно ли построить топологическую сортировку.
3. Найти критический путь — максимальную по длительности последовательность задач.
4. Определить минимальную продолжительность проекта.
5. Рассчитать ранние и поздние начала каждой задачи, выявить задачи, которые можно выполнять параллельно.
6. Вычислить временной резерв для каждой задачи (разность между поздним и ранним стартом).

Вариант 3. Выбор оптимального маршрута уборки улиц города.

Постановка:

В муниципальном районе "Центральный" необходимо организовать механизированную уборку улично-дорожной сети. Район включает несколько ключевых улиц, соединяющих крупные площади и перекрестки; каждая улица имеет определённую длину. Спецтехника выезжает с базы, расположенной на площади Мира, и должна пройти все улицы с минимальными повторениями и затратами времени — оптимально, если по каждой улице будет совершён ровно один проход. Планирование затрудняется, если сеть не позволяет эйлеров обход, и тогда следует минимизировать двойные проходы по улицам.

Пример:

В сети участвуют следующие улицы:

- ул. Ленина (1.2 км) — соединяет площадь Мира и ул. Советскую;
- ул. Советская (0.8 км) — соединяет ул. Ленина с ул. Пушкина;
- ул. Пушкина (1.5 км) — соединяет ул. Советская с площадью Победы;
- пр. Гагарина (2.1 км) — соединяет площадь Мира с площадью Победы;
- ул. Мира (0.6 км) — соединяет ул. Ленина и пр. Гагарина;
- ул. Парковая (0.9 км) — соединяет ул. Пушкина и пр. Гагарина.

Входные данные:

Перечень улиц, точек пересечений (площадей и перекрёстков), а также длины каждой улицы (см. пример выше).

Требуется:

1. Построить граф улично-дорожной сети (вершины — пересечения и площади, рёбра — улицы с длинами).
2. Определить степень каждой вершины и проверить возможность эйлерова обхода (условия существования эйлерова цикла/пути).
3. Найти эйлеров цикл (если есть) или оптимальный эйлеров путь.
4. Если эйлеров путь невозможен, определить маршрут с минимальным количеством повторных проходов по улицам.
5. Рассчитать общую длину наилучшего маршрута с учётом повторений.
6. Указать стартовую и конечную точки маршрута (база на площади Мира).

Вариант 4. Нахождение кратчайшего маршрута доставки курьера.

Постановка:

Курьеру службы “Доставка+” поручено развести заказы по N клиентским адресам за одну смену, начиная и заканчивая работу в офисе компании. Между всеми адресами и офисом известны точные расстояния по автодорогам (с учетом городских развязок). Необходимо минимизировать общий пробег транспортного средства и тем самым затраты на топливо и время. Важно сравнить результат оптимального маршрута с более простым жадным методом “ближайшего соседа”, а также оценить экономический эффект оптимизации.

Пример:

Курьеру даны следующие адреса:

- Офис (ул. Центральная, 15);
- ул. Московская, 23 — от офиса 2.1 км;
- пр. Ленина, 45 — от офиса 1.8 км, от ул. Московской 1.2 км;
- ул. Садовая, 12 — от офиса 3.2 км, от ул. Московской 2.8 км, от пр. Ленина 1.9 км;
- ул. Парковая, 8 — от офиса 2.5 км, от ул. Московской 1.5 км, от пр. Ленина 2.2 км, от ул. Садовой 1.1 км;
- ул. Заречная, 34 — от офиса 4.1 км, от ул. Московской 3.3 км, от пр. Ленина 2.7 км, от ул. Садовой 1.8 км, от ул. Парковой 2.0 км.

Входные данные:

Список всех адресов и полная матрица расстояний между каждой парой точек.

Требуется:

1. Построить полный взвешенный граф по этим данным.
2. Решить задачу коммивояжера — найти кратчайший замкнутый маршрут, проходящий через все точки ровно по одному разу.
3. Применить жадный алгоритм “ближайшего соседа”, сравнить длину с оптимальной.
4. Указать итоговую длину оптимального маршрута и конкретную последовательность посещения адресов.
5. Оценить потенциальную экономию топлива и рабочего времени курьера по сравнению с произвольным или неэффективным маршрутом.

Вариант 5. Анализ транспортной доступности районов города.

Постановка:

В городе "Новосибирск" городской совет анализирует транспортную доступность между N районами, связанными маршрутами общественного транспорта: автобусами, метро и трамваями. Каждый маршрут связывает две конкретные точки и имеет свой идентификатор (например, автобус №5 связывает Центральный и Кировский, метро — Центральный и Ленинский). Вопрос состоит в выявлении уязвимых мест транспортной сети: существуют ли районы, до которых нельзя добраться общественным транспортом, как распределена центральность, какие соединения критичны для связности всей сети, а какие — избыточны.

Пример:

Сеть маршрутов включает следующие маршруты:

- Центральный ↔ Кировский (автобус №5);
- Центральный ↔ Ленинский (метро, линия 1);
- Кировский ↔ Советский (автобус №12);
- Ленинский ↔ Октябрьский (автобус №7);
- Советский ↔ Дзержинский (трамвай №4);
- Октябрьский ↔ Заельцовский (автобус №15);
- Дзержинский ↔ Железнодорожный (автобус №23);
- Заельцовский ↔ Железнодорожный (автобус №31).

Входные данные:

Полная таблица районов и связей между ними с указанием вида транспорта.

Требуется:

1. Построить неориентированный граф транспортных связей между районами.
2. Проверить связность графа методом обхода в глубину (DFS), выявить недоступные районы, если таковые имеются.
3. Найти кратчайшие пути между всеми парами районов.
4. Определить центральный район по минимальному значению максимального кратчайшего расстояния до остальных (эксцентриситет).
5. Выявить мостовые соединения (ребра, разрыв которых нарушает связность графа).

6. Рассчитать диаметр транспортной сети города (максимальное кратчайшее расстояние между районами).

Вариант 6. Анализ отказоустойчивости корпоративной сети.

Постановка:

В компании ООО "ИнфоТех" эксплуатируется корпоративная сеть, объединяющая рабочие станции, серверы, сетевые коммутаторы и маршрутизаторы. Сеть построена по резервируемой топологии: рабочие станции сгруппированы по коммутаторам, сервера подключены к отдельному коммутатору, вся инфраструктура связана через два маршрутизатора с резервированием. Цель анализа — определить уровень отказоустойчивости сети: какие узлы и соединения являются критическими, сколько устройств или каналов требуется отключить для нарушения работы сети, а также предложить возможные усовершенствования топологии.

Пример:

В корпоративной сети присутствуют:

- 15 рабочих станций (номера WS-01–WS-15);
- 3 сервера: файловый, почтовый, web-сервер;
- 4 коммутатора (Switch-01–Switch-04);
- 2 роутера (Router-01, Router-02);
- 1 основной шлюз в Интернет (Gateway-01).

Топологические связи:

- каждый коммутатор обслуживает 3–4 рабочие станции (например, Switch-01: WS-01–WS-04);
- все серверы подключены к Switch-01;
- все коммутаторы соединены с маршрутизаторами для резервирования: Router-01 ↔ Switch-01, Switch-02, Router-02 ↔ Switch-03, Switch-04;
- Gateway-01 связан с обоими маршрутизаторами.

Входные данные:

Структурированная таблица узлов (устройств), типов устройств и явных связей между ними (как в примере выше).

Требуется:

1. Построить неориентированный граф сетевой топологии с типами устройств.
2. Определить вершинную связность сети (минимальное количество узлов, удаление которых нарушает связность всей сети).

3. Определить рёберную связность (минимальное количество каналов для разделения сети на несвязанные части).
4. Выделить точки сочленения (узлы, отказ которых разделяет сеть).
5. Найти мостовые соединения (критические линии связи).

Вариант 7. Составление графика работы сотрудников.

Постановка:

Ресторан “Гурман” открыт ежедневно и обслуживает гостей в три смены (утреннюю, дневную, ночную). Всего задействовано 12 сотрудников различных профессий: повара, официанты, уборщики и администраторы. Каждый сотрудник может работать не более 5 дней в неделю, при этом для каждой смены требуется строго определённый состав команды: 1 повар, 2 официанта, 1 уборщик, 1 администратор. Для поваров и администраторов есть ограничения по доступным сменам: например, поварам запрещено работать ночью, а администраторы — только днём. Ресторану требуется каждый день соблюдать все кадровые ограничения и при этом равномерно распределить нагрузку среди сотрудников, а также предусмотреть резервные варианты на случай болезни ключевого работника.

Пример:

Штат сотрудников:

- повара: Антонов, Белов, Васильев, Гришин (не могут работать в ночную смену);
- официанты: Дмитриев, Егоров, Жуков, Зайцев;
- уборщики: Иванов, Козлов;
- администраторы: Львов, Михайлов (работают только в дневную смену).

Входные данные:

Список всех сотрудников (разделённых по профессиям) и ограничений по рабочим сменам для каждого, а также требования к составу смены.

Требуется:

1. Сформировать двудольный граф “сотрудники—смены” с учётом всех ограничений (работоспособность, максимальная нагрузка, запреты).
2. Найти максимальное паросочетание для оптимального распределения сотрудников по сменам.
3. Проверить, обеспечивает ли график выполнение всех требований к укомплектованию смен.
4. Сформировать недельный график работы для каждого сотрудника (календарь занятости).
5. Рассчитать загрузку каждого сотрудника (количество смен за неделю).

Вариант 8. Анализ молекулярной структуры.

Постановка:

В химической лаборатории исследуют структуру молекулы кофеина ($C_8H_{10}N_4O_2$). Молекула состоит из углерода, водорода, азота и кислорода, образует сложную систему ковалентных связей с двумя конденсированными кольцами (пиримидиновым и имидазольным). Химикам важно проверить, насколько структура соответствует классическим химическим правилам: правильная ли валентность атомов, равномерно ли распределены связи между элементами, какова цикличность молекулы и можно ли её плоско изобразить без пересечений связей (планарность графа).

Пример:

Состав кофеина:

- 8 атомов углерода;
- 10 атомов водорода;
- 4 атома азота;
- 2 атома кислорода.

Всего: 24 атома, 25 ковалентных связей.

Структура колец:

- Пиримидиновое кольцо: 6-членное (атомы С и N);
- Имидазольное кольцо: 5-членное (атомы С и N), конденсировано с пиримидином.

Входные данные:

Список атомов с указанием всех связей в виде списка пар (или таблицы), а также информация, какие связи образуют кольца.

Требуется:

1. Построить молекулярный граф (вершины — атомы, рёбра — ковалентные связи).
2. Проверить регулярность (одинаковость степеней атомов одного типа).
3. Найти все циклы и определить их длины (размеры молекулярных колец).
4. Вычислить степень каждого атома и проверить соответствие валентности.
5. Оценить возможность планарного представления структуры (графа).
6. Выявить участки (например, группы атомов), где нарушается классическая химическая регулярность или валентность.

Вариант 9. Размещение компонентов на печатной плате.

Постановка:

Компания-разработчик систем “УмныйДом-2025” проектирует печатную плату для микроконтроллерного устройства управления. На плате размещаются ключевые электронные компоненты: микропроцессор, различные микросхемы памяти, разъемы, резисторы, конденсаторы. Электрические соединения образуют сложную сеть дорожек, которые должны учитывать технологические ограничения (минимальные расстояния между компонентами и проводниками, используемые корпуса, тепловые и электромагнитные требования). Важно определить, возможно ли расположить все соединения и элементы на одной плате без пересечений дорожек, сколько слоев потребуется для разводки, как оптимально сгруппировать компоненты с точки зрения монтажа, обслуживания и безопасности.

Пример:

Компоненты:

- Микропроцессор U1 (ARM Cortex-M4, QFP-64, 8×8 мм);
- Flash память U2 (SOIC-8, 5×4 мм);
- EEPROM U3 (SOIC-8, 5×4 мм);
- RAM U4 (TSOP-28, 11×4 мм);
- SD-карта: U5 (разъем, 15×11 мм);
- Буферные памяти U6-U9 (SOT-23, 3×3 мм);
- Резисторы R1–R12 (2×1 мм);
- Конденсаторы C1–C6 (2×1 мм).

Требования к соединениям:

- 15 дорожек с необходимыми трассировками (питание, шины данных, адреса, спецсигналы);
- минимальное расстояние между компонентами: 0.5 мм;
- ширина дорожки: 0.2 мм; между дорожками: 0.2 мм; диаметр переходного отверстия (via): 0.3 мм.

Тепловые ограничения:

- U1 выделяет до 200 мВт тепла, требуется заливка;
- минимальное расстояние от U1 до термочувствительных компонентов: 5 мм.

Электромагнитные ограничения:

- высокочастотные сигналы (CLK_EXT) — экранировать; длина до 15 мм.

Входные данные:

Таблица компонентов с размерами, типами корпусов, требованиями по соединениям; список электрических дорожек с назначением и ограничениями, тепловые и электромагнитные параметры.

Требуется:

1. Построить граф соединений (вершины — компоненты, рёбра — электрические связи, каждая с типом сигнала и требованиями по разводке).
2. Проверить планарность графа (возможно ли разместить все соединения на одном слое без пересечений).
3. Если планарность невозможна — вычислить минимальное количество пересечений/слоев для полного размещения.
4. Осуществить оптимальную раскладку компонентов в пределах площади платы 60×40 мм с учетом тепловых и монтажных ограничений, доступности для обслуживания, группировок по функционалу.

Вариант 10. Размещение станции скорой помощи.

Постановка:

Район заданной площади включает N населённых пунктов различной численности: сёла и деревни, разбросанные по территории с известными координатами. Необходимо определить оптимальное место для размещения единственной станции скорой медицинской помощи так, чтобы среднее время доезда к пациенту было минимальным с учетом численности населения каждого пункта (плотность потенциальных вызовов). Анализ охватывает расчёт расстояний между всеми населёнными пунктами, выделение центральной точки района по критериям графовой теории, а также оценку доступности медпомощи для каждого поселения.

Пример:

Район площадью 15×12 включает следующие населённые пункты с координатами и населением:

- Село Центральное: координаты (7.5, 6), население 2500 чел;
- Село Северное: координаты (3, 10), население 1200 чел;
- Село Южное: координаты (11, 2), население 1800 чел;
- Село Западное: координаты (2, 5), население 900 чел;
- Село Восточное: координаты (13, 7), население 1600 чел;
- Деревня Лесная: координаты (5, 8), население 400 чел;
- Деревня Полевая: координаты (9, 4), население 600 чел;
- Деревня Речная: координаты (6, 2), население 350 чел.

Входные данные:

Список координат и численности для каждого населённого пункта.

Требуется:

1. Построить взвешенный граф поселений, где веса рёбер — расстояния.
2. Вычислить эксцентриситет каждого пункта для возможного размещения станции.
3. Найти центр графа (точку с минимальным максимальным расстоянием до остальных узлов).
4. Рассчитать средневзвешенное время доезда (с учетом численности населения каждого пункта).
5. Предложить оптимальное место для станции скорой помощи.

6. Проанализировать доступность медпомощи для каждого населённого пункта, указать возможные проблемные зоны.

Вариант 11. Анализ социальной сети университета.

Постановка:

На факультете университета развернута внутренняя социальная сеть для N студентов. Каждый студент может быть связан с другими по разным типам отношений: учебные группы, соседи по общежитию, участники кружков и секций, случайные знакомства. Система должна выявлять лидеров по связям, анализировать плотность и распределение контактов, структуру кластеров и “малые миры”. Необходимо сравнить реальные параметры сети с теоретическими моделями (например, Эрдёша-Реньи, предпочтительного присоединения, “small world”).

Пример:

Социальная сеть студентов факультета (120 человек) имеет следующие характеристики:

- средняя степень узла: 8.5 связей;
- диаметр сети: 5 (максимальное "расстояние рукопожатий");
- средний путь между студентами: 2.8 связей;
- количество треугольных связей: 340;
- коэффициент кластеризации: 0.42.

Распределение студентов по курсам:

- первый курс: 35 человек;
- второй курс: 32 человека;
- третий курс: 28 человек;
- четвертый курс: 25 человек.

Активность в социальных сетях:

- очень активные (более 15 связей): 12 студентов;
- активные (10-15 связей): 28 студентов;
- умеренно активные (5-9 связей): 45 студентов;
- малоактивные (2-4 связи): 25 студентов;
- изолированные (0-1 связь): 10 студентов.

Типы связей:

- одноклассники: 45% всех связей;
- соседи по общежитию: 25% связей;
- участники кружков и секций: 20% связей;

- случайные знакомства: 10% связей.

Входные данные:

Таблица всех студентов, числа и типов связей, параметры сети (средняя степень, диаметр, кластеризация и др.).

Требуется:

1. Проанализировать структурные свойства студенческой сети (количество, типы связей, распределение активности).
2. Вычислить средний диаметр сети (среднее расстояние между всеми парами студентов).
3. Найти наиболее центральных студентов (лидеры мнений по связности).
4. Определить плотность сети и сравнить её с теоретическими моделями (Эрдёша-Реньи, модель малого мира, предпочтительное присоединение).
5. Выделить кластеры (сообщества) студентов с высокой внутренней связанностью и визуализировать их (группы по интересам, курсы).

Вариант 12. Анализ популярности веб-страниц.

Постановка:

Корпоративный сайт компании состоит из N страниц, связанных сложной системой внутренних гиперссылок: главная страница ведет в основные разделы, сервисные и продуктовые страницы включают навигацию между собой, а в нижнем колонтитуле каждой страницы есть переход на “Политику конфиденциальности”. Для улучшения юзабилити и продвижения (SEO) требуется провести всесторонний графовый анализ структуры сайта: выявить “стержневые” страницы, просчитать степень их популярности (по входящим ссылкам), определить важнейшие источники трафика и “тупиковые” ресурсы, а также вычислить PageRank и ранжировать страницы по важности.

Пример:

Структура внутренних ссылок:

1. Главная страница ссылается на: О компании, Услуги, Продукты, Новости, Контакты, Каталог товаров, Отзывы клиентов, Вакансии.
2. “Услуги” ведут на: Продукты, Каталог товаров, Прайс-лист, Отзывы клиентов, Контакты, Техподдержку.
3. “Продукты” связаны с: Каталог товаров, Прайс-лист, Отзывы клиентов, Контакты.
4. “Каталог товаров” ссылается на: Продукты, Прайс-лист, Отзывы клиентов, Контакты, Техподдержку.

Кроме этого, на каждой странице указана ссылка в футере на “Политику конфиденциальности”. Также для каждой страницы предоставлены данные по уникальным посетителям и времени пребывания (например, Каталог товаров — 9500 посетителей, 240 с).

Входные данные:

Таблица всех страниц сайта, внутренних ссылок между ними, статистика посещаемости (время на странице, количество уникальных посетителей).

Требуется:

1. Построить ориентированный граф структуры сайта (вершины — страницы, рёбра — ссылки).

2. Для каждой страницы вычислить полу-степени входа и выхода (количество входящих/исходящих ссылок).
3. Определить наиболее популярные страницы по числу входящих ссылок.
4. Найти страницы-источники (максимум исходящих ссылок).
5. Рассчитать коэффициент PageRank для каждой страницы сайта и ранжировать их по важности.
6. Выделить тупиковые страницы (без исходящих ссылок), анализировать их роль в навигации.

Вариант 14. Анализ экономических связей.

Постановка:

В одном регионе функционирует N компаний из различных отраслей: производственные предприятия, торговые компании, транспортные и строительные фирмы, IT-компании, банки. Между компаниями заключено множество разнородных договоров: поставки товаров и сырья, подрядные работы, кредитные соглашения. Региональные органы и крупные участники рынка заинтересованы проанализировать устойчивость бизнес-структуры: выявить центральные и уязвимые организации, определить изолированные группы и оценить уровень межотраслевого взаимодействия. Важно также количественно оценить, какие компании системно значимы для целостности экономической сети.

Пример:

Состав компаний:

- 15 производственных предприятий;
- 12 торговых компаний;
- 8 транспортных компаний;
- 7 строительных фирм;
- 5 IT-компаний;
- 3 банка.

Типы связей:

- 95 договоров поставки продукции (например, между производственными и торговыми компаниями);
- 45 подрядных договоров (например, заказы строительным и транспортным фирмам);
- 23 кредитных договора (участвуют банки).

Входные данные:

Список всех компаний с указанием отрасли; таблица всех договоров с типом (поставка, подряд, кредит).

Требуется:

1. Построить мультиграф экономических связей (вершины — компании, рёбра — договоры с типом).

2. Найти компоненты связности — выявить изолированные кластеры компаний и независимые группы.
3. Определить центральные компании (по количеству и разнообразию связей, их типам).
4. Проанализировать межотраслевые связи и структуру взаимодействия между секторами.
5. Определить компании, чье банкротство может нарушить связность сети (системно значимые узлы).

Вариант 15. Поиск общих партнёров компаний.

Постановка:

Две крупные компании региона ведут активную деловую деятельность с разными фирмами-партнёрами. Однако руководство заинтересовано выявить — есть ли у компаний общие контрагенты, как велико пересечение партнерских сетей и возможно ли развитие новых направлений сотрудничества через существующие деловые связи. Анализ включает построение графов партнёрских связей для каждой компании, формальное определение пересечения и объединения этих сетей, а также оценку уровня “близости” бизнес-интересов.

Пример:

Партнёры компании “АльфаТех”:

- ООО "БетаСтрой";
- ЗАО "ГаммаТорг";
- ИП "ДельтаЛогистика";
- ООО "ЭпсилонПром";
- ЧП "ЗетаИнвест".

Партнёры компании “МегаКорп”:

- ООО "БетаСтрой";
- ЗАО "ГаммаТорг";
- ООО "ТетаФинанс";
- ИП "КаппаМаркет";
- ООО "ЛямбдаСервис".

Входные данные:

Для каждой компании — полный список наименований партнёров.

Требуется:

1. Построить два графа партнёрских связей: для “АльфаТех” и для “МегаКорп” (вершины — компании и все их партнёры, рёбра — наличие деловых отношений).
2. Выполнить операцию пересечения графов (найти общих партнёров).
3. Найти и перечислить имена всех общих партнёров.

4. Выполнить объединение графов, сформировав общую сеть деловых связей.
5. Проанализировать возможности расширенного сотрудничества через общих партнёров.

Вариант 16. Размещение логистических складов.

Постановка:

Торговая сеть “Продмаг” обслуживает N магазинов, расположенных в радиусе M км друг от друга. Каждый магазин характеризуется координатой и объёмом ежемесячных отгрузок. Основная задача — минимизировать транспортные расходы путём оптимального размещения одного или нескольких логистических складов: найти такие точки для склада, чтобы суммарные затраты на доставку товаров во все магазины были минимальны, при этом учесть распределение грузооборота, стоимость километра перевозки и ограничения на площади и аренду складских помещений. Дополнительно требуется рассчитать экономический эффект и зоны обслуживания складов для разных моделей размещения: одного, двух или трёх складов.

Пример:

Торговая сеть "Продмаг" обслуживает 15 магазинов в радиусе 80 км. Координаты магазинов:

- Магазин №1: (10, 15), объём поставок 200 т/мес;
- Магазин №2: (25, 30), объём поставок 350 т/мес;
- Магазин №3: (45, 20), объём поставок 180 т/мес;
- Магазин №4: (35, 45), объём поставок 280 т/мес;
- Магазин №5: (55, 35), объём поставок 320 т/мес;
- Магазин №6: (12, 40), объём поставок 150 т/мес;
- Магазин №7: (65, 25), объём поставок 240 т/мес;
- Магазин №8: (30, 12), объём поставок 190 т/мес;
- Магазин №9: (8, 25), объём поставок 160 т/мес;
- Магазин №10: (48, 55), объём поставок 220 т/мес;
- Магазин №11: (70, 40), объём поставок 200 т/мес;
- Магазин №12: (18, 60), объём поставок 130 т/мес;
- Магазин №13: (40, 8), объём поставок 170 т/мес;
- Магазин №14: (22, 50), объём поставок 210 т/мес;
- Магазин №15: (60, 15), объём поставок 260 т/мес.

Общий грузооборот сети – 3265 т/мес.

Характеристики торговой сети:

- стоимость транспортировки – 12 руб/км·т;
- стоимость аренды склада – 800 руб/м² в месяц;

- площадь склада для хранения 100 тонн – 200 м²;
- средняя скорость доставки – 45 км/ч;
- рабочее время доставки – 8 часов в день;
- количество рабочих дней в месяце – 22 дня.

Входные данные:

Список магазинов (координаты, объём доставки), данные о ценах, площадях, временные характеристики.

Требуется:

1. Построить взвешенный граф магазинов, где веса рёбер — грузооборот между точками.
2. Рассчитать расстояния между всеми магазинами.
3. Найти оптимальную точку расположения единого склада, минимизирующую затраты на доставку.
4. Разделить магазины на 2 и 3 группы с географическим и грузовым анализом, рассчитать координаты для каждого склада.
5. Указать зоны обслуживания (какие магазины какой склад обслуживает).
6. Посчитать экономический эффект по формуле: Месячные транспортные затраты = $\Sigma(\text{расстояние} \times \text{объём} \times \text{стоимость км/т} \times 2)$.

Вариант 17. Маршрут контролёра на производстве

Постановка:

В крупном производственном цехе размещены N технологических линий, соединённых сетью коридоров и переходов фиксированной длины. Работник-контролёр должен ежедневно обходить все линии для инспекции оборудования, двигаясь по коридорам так, чтобы все переходы были пройдены с минимальными затратами времени, не повторяя лишний раз одни и те же участки. Если невозможно пройти все переходы ровно один раз (нет эйлера цикла), следует составить маршрут с минимальным числом повторов. Также требуется оценить общее время обхода и рациональность маршрута.

Пример:

Производственный цех содержит 8 технологических линий, соединённых переходами:

- Линия 1 $\leftrightarrow$ Линия 2 (50 м);
- Линия 2 $\leftrightarrow$ Линия 3 (30 м);
- Линия 3 $\leftrightarrow$ Линия 4 (40 м);
- Линия 4 $\leftrightarrow$ Линия 5 (35 м);
- Линия 5 $\leftrightarrow$ Линия 6 (25 м);
- Линия 6 $\leftrightarrow$ Линия 7 (45 м);
- Линия 7 $\leftrightarrow$ Линия 8 (55 м);
- Линия 8 $\leftrightarrow$ Линия 1 (60 м) — замыкающий переход.

Входные данные:

Перечень всех технологических линий, список и длины переходов между ними, скорость движения контролёра.

Требуется:

1. Построить граф техлиний: вершины — линии, рёбра — переходы с длинами.
2. Проверить, возможно ли построить эйлеров обход (цикл/путь) по всем переходам.
3. Если да, найти такой маршрут; если нет — составить оптимальный с минимальными повторениями.
4. Рассчитать общую длину маршрута обхода.
5. Определить время обхода при скорости движения контролёра 3 км/ч.

Вариант 18. Анализ изоморфизма организационных структур

Постановка:

В двух крупных организациях внедряются схемы управления, отличающиеся по названиям должностей и ветвям подчинения. Руководство хочет понять: можно ли перейти от одной организационной схемы к другой без изменения характера подчинённых связей, а исключительно через переименование и перестановку позиций? По сути, требуется провести анализ структурного изоморфизма двух оргструктур, выявить соответствия между должностями, сравнить глубину и ширину иерархий, и формализовать различия или совпадения в терминах теории графов.

Пример:

Организация А:

Директор

Замдиректор по производству

Мастер цеха

Старший технолог

Главбух

Экономист

Организация Б:

CEO

Production Manager

Shop Supervisor

Chief Technologist

Financial Manager

Accountant

Входные данные:

Полные схемы подчинения (список должностей и связей) для обеих организаций.

Требуется:

1. Построить два графа оргструктур (вершины — должности, рёбра — подчинение).
2. Выяснить, являются ли графы изоморфными (можно ли перевести одну структуру в другую только переименованием/перестановками без изменения структуры связей).
3. Если да, указать взаимно-однозначное соответствие между позициями.
4. Если нет, явно описать различия (разная глубина, разветвленность, наличие дополнительных позиций и т.д.).

Вариант 19. Оптимизация системы видеонаблюдения: задача о минимальном доминирующем множестве

Постановка:

В крупном производственном или складском помещении требуется добиться полного видеоконтроля за всеми зонами. Каждая камера, установленная в определённой точке, может охватывать несколько зон одновременно (то есть обладает зоной видимости). Стоимость оборудования и монтажных работ велика, поэтому задача сводится к поиску минимального количества камер и их позиций, необходимых для покрытия всех территорий. Необходимо подобрать схему размещения так, чтобы все рабочие зоны попадали в поле зрения хотя бы одной камеры, а избыточное перекрытие строго минимизировалось.

Пример:

Пример схемы склада:

- возможные точки для размещения камер: P1, P2, P3, P4, P5;
- зоны (участки): Z1, Z2, Z3, Z4, Z5, Z6;
- зона видимости камер:
 - для P1: Z1, Z2, Z3;
 - для P2: Z2, Z4;
 - для P3: Z3, Z4, Z5;
 - для P4: Z5, Z6;
 - для P4: Z5, Z6.

Входные данные:

Список всех возможных точек установки камер, для каждой — перечень зон, которые она перекрывает.

Требуется:

1. Построить двудольный граф: одна доля — точки установки камер, другая — зоны, рёбра — есть видимость этой зоны из данной точки.
2. Найти минимальное доминирующее множество точек: такой набор камер, чтобы все зоны были покрыты хотя бы одной камерой.
3. Перечислить номера и позиции выбранных камер.
4. Составить оптимальную схему размещения и указать для каждой зоны, какая(ие) камера(ы) её контролируют.

Вариант 20. Распределение задач между проектными группами: задача о сбалансированной нагрузке

Постановка:

В крупной IT-компании организовано несколько проектных групп для разработки различных модулей единого программного продукта. Каждая задача по доработке или интеграции может быть выполнена одной или несколькими группами, а некоторые задачи требуют кооперации двух и более команд. Важно не только назначить группы на задачи с учетом специализации, но и добиться того, чтобы общая нагрузка (количество задач на группу) была сбалансированной — ни одна команда не перегружена, все задачи закрыты в срок.

Пример:

Группы: G1, G2, G3, G4.

Задачи: T1–T8.

- T1: может выполнять G1, G3;
- T2: может выполнять G1, G2;
- T3: только G2;
- T4: G2, G4;
- T5: G3, G4;
- T6: G1, G4;
- T7: G2, G3;
- T8: только G4.

Число задач на группу:

- G1: T1, T2, T6;
- G2: T2, T3, T4, T7;
- G3: T1, T5, T7;
- G4: T4, T5, T6, T8.

Входные данные:

Список проектных групп и задач, а также для каждой задачи — список групп, которые могут её выполнить.

Требуется:

1. Построить двудольный граф “группы–задачи”: вершины — группы и задачи, рёбра — может выполнить.
2. Найти допустимое распределение задач — каждой задаче назначить группу (или группы) так, чтобы покрыть все задачи.
3. Выполнить балансировку: минимизировать разницу между максимальным и минимальным числом задач, назначенных на одну группу.
4. Предложить схему, при которой нет перегрузки, каждая задача делается либо одной, либо несколькими назначенными группами.

Вариант 21. Анализ мостов и уязвимых связей в компьютерной или транспортной сети

Постановка:

В сложной распределённой сети (городской транспортной или корпоративной компьютерной) необходимо выявить такие рёбра ("мосты" или критические каналы), отключение которых сразу же разделит сеть на две или более изолированные части. Для долгосрочной устойчивости важно понимать, какие соединения требуют резервирования, а какие — избыточны, чтобы оптимизировать затраты на поддержание инфраструктуры. В качестве расширения задачи можно оценить влияние одновременного выхода из строя нескольких каналов.

Пример:

Граф транспортной сети:

- вершины: A, B, C, D, E, F;
- связи:
A–B, B–C, C–D, D–E, E–F, F–A (кольцо);
B–E, C–F (дополнительные связи).

Если удалить рёбра B–C или D–E, сеть останется связной за счёт других каналов.

Если удалить A–B или F–A — и при этом кольцо разомкнуто — сеть распадётся на две части.

Входные данные:

Список узлов и всех связей (ребер) между ними.

Требуется:

1. Определить все мосты (ребра, удаление которых увеличивает число компонент связности).
2. Для каждой критической связи указать, какие части сети она соединяет.
3. Оценить критичность каналов с точки зрения их длины, стоимости или интенсивности трафика (если есть).